

优先级队列 堆排序

选取

❖ 在 selectionSort() 中

将 **U** 替换为 **H**...

❖ J. Williams, 1964

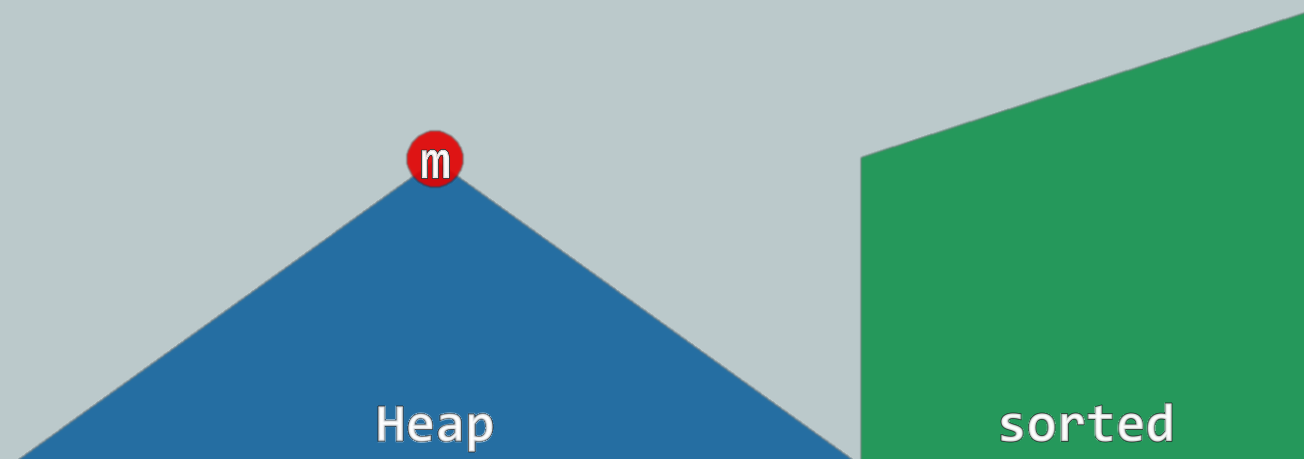
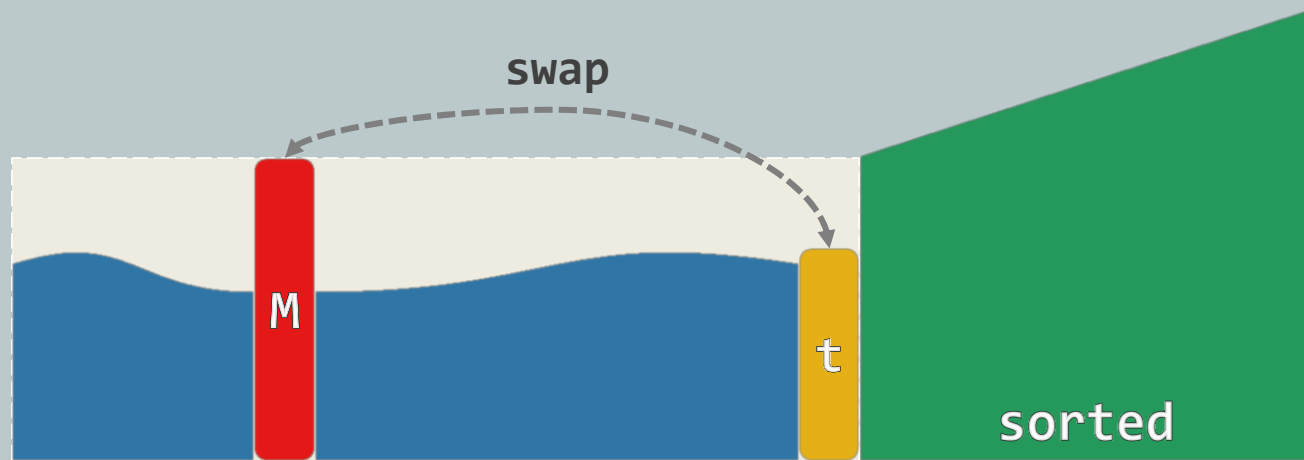
初始化 : heapify(), $O(n)$

迭代 : delMax(), $O(\log n)$

不变性 : $H \leq S$

❖ $O(n) + n \times O(\log n)$

= $O(n \log n)$



就地

❖ 在物理上

完全二叉堆即是向量

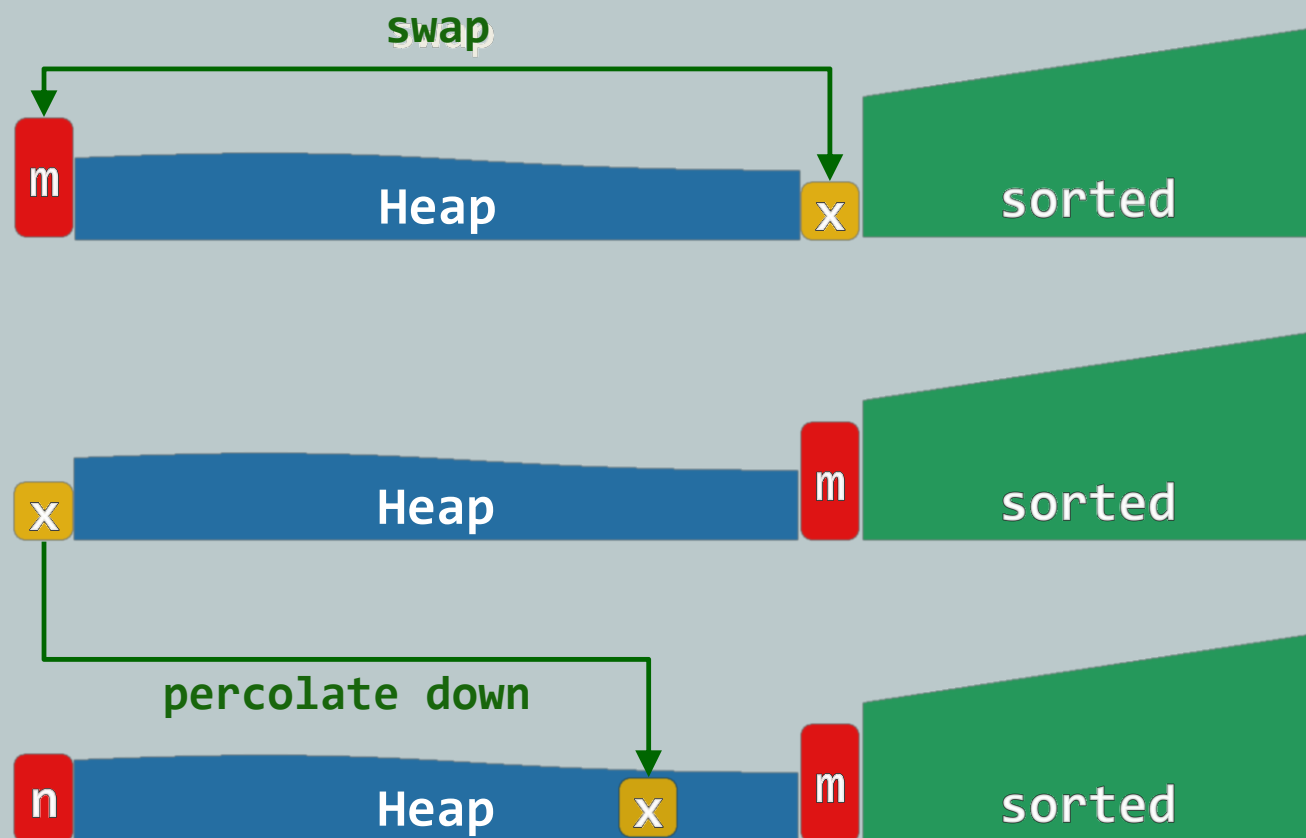
❖ 既然此前有：

$$- m = H[0]$$

$$- x = H[n - 1]$$

不妨随即就：

$$- \text{swap}(m, x) = H.\text{insert}(x) + S.\text{insert}(m)$$



实现

```
template <typename T> //对向量区间[lo, hi)做就地堆排序
```

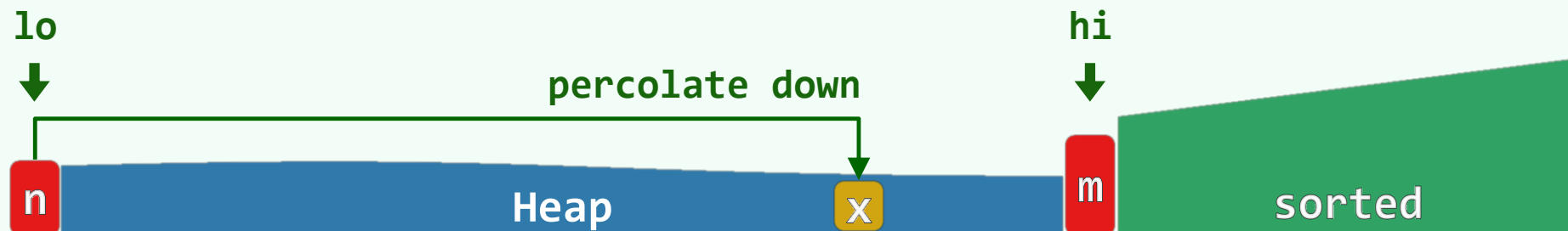
```
void Vector<T>::heapSort( Rank lo, Rank hi ) {
```

```
    T* A = _elem + lo; Rank n = hi - lo; heapify( A , n ); //待排序区间建堆,  $O(n)$ 
```

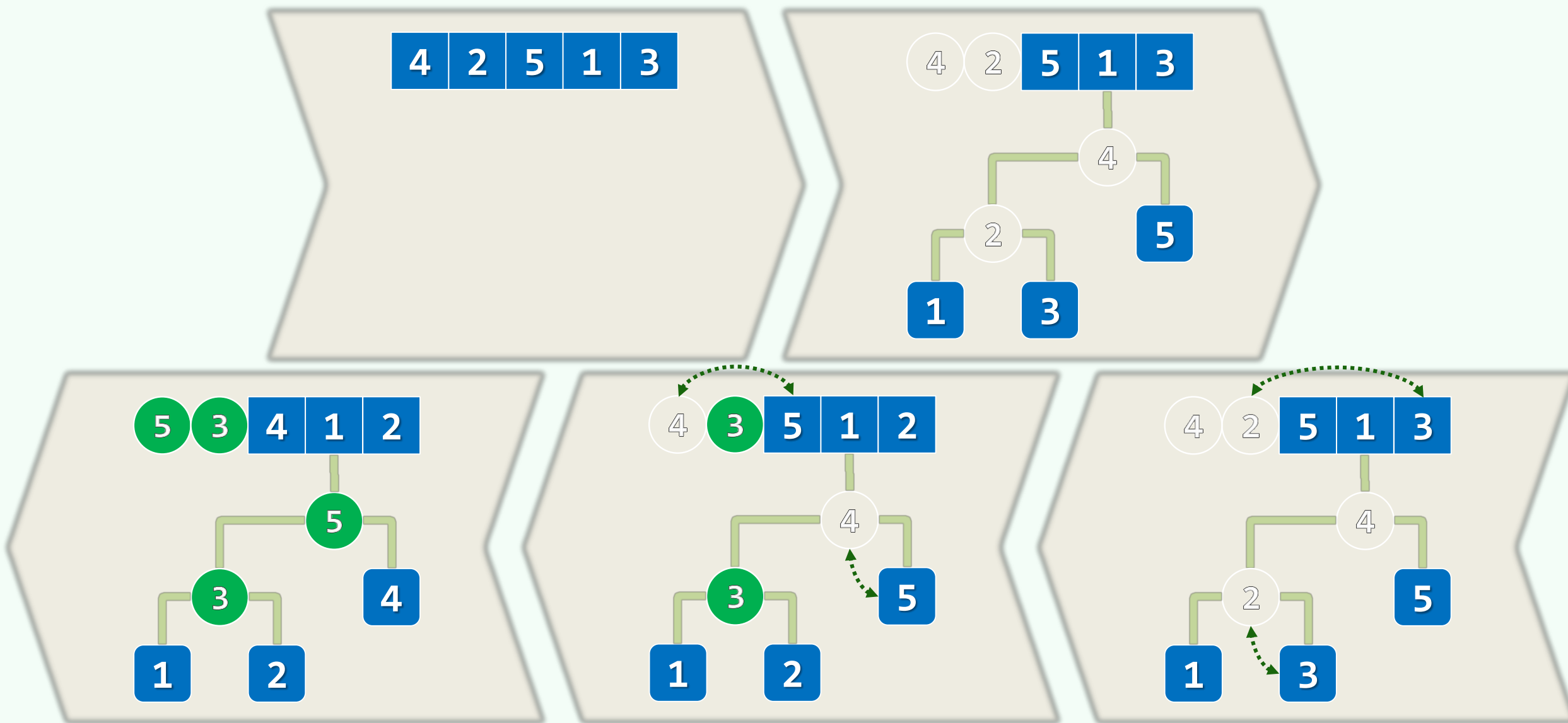
```
    while ( 0 < --n ) //反复地摘除最大元并归入已排序的后缀, 直至堆空
```

```
        { swap( A[0], A[n] ); percolateDown( A, n, 0 ); } //堆顶与末元素对换后下滤
```

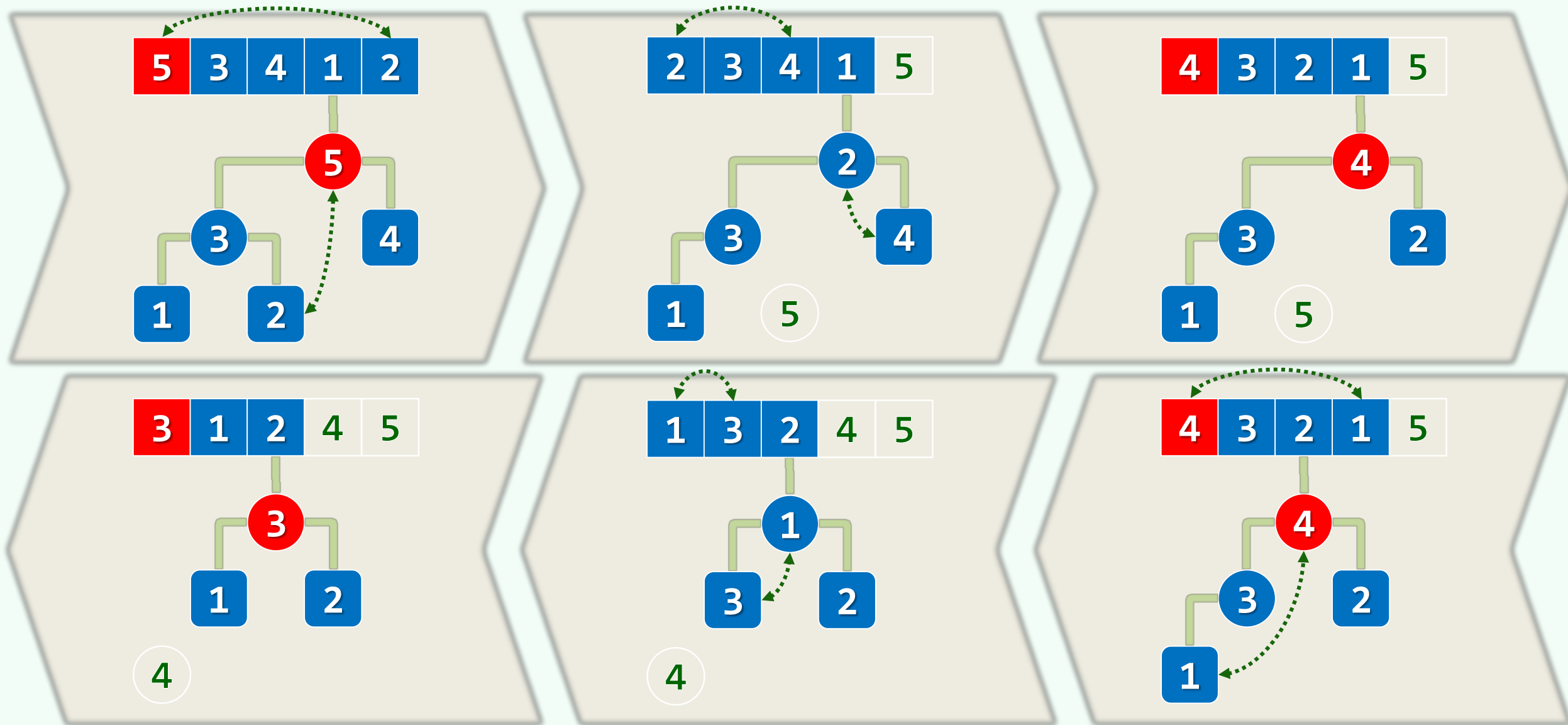
```
}
```



实例：建堆



实例：选取 + 调整(1/2)



实例：选取 + 调整(2/2)

1 2 3 4 5

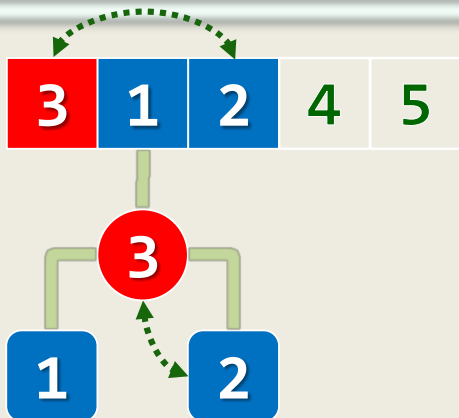
1 2 3 4 5

1 2 3 4 5

1

1

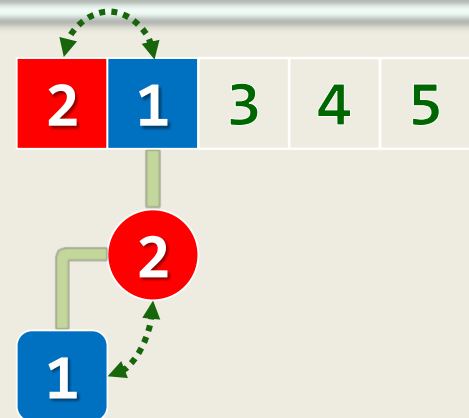
2



2 1 3 4 5

2

3



优先级队列

锦标赛树：胜者树

锦标赛树

❖ Tournament Tree

完全二叉树

- 叶节点：待排序元素（选手）
- 内部节点：孩子中的胜者

❖ `create()` // $O(n)$

`remove()` // $O(\log n)$

`insert()` // $O(\log n)$

❖ **树根**总是**全局冠军**：类似于堆顶

❖ 内部结点各对应于一场比赛的胜者——重复存储



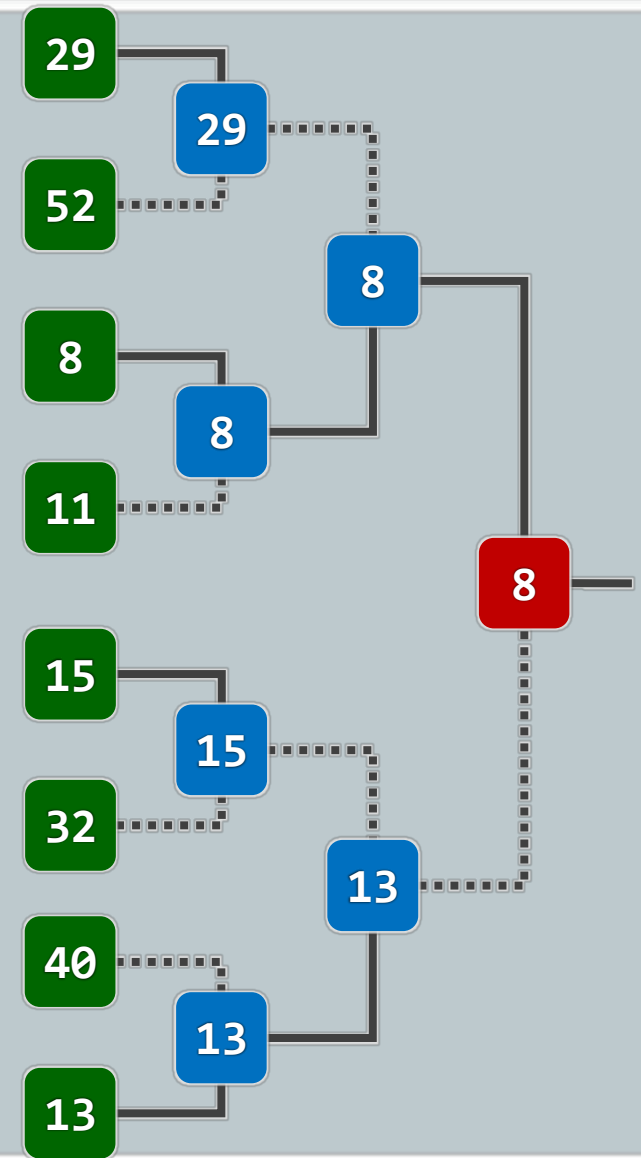
用以排序

❖ Tournamentsort()

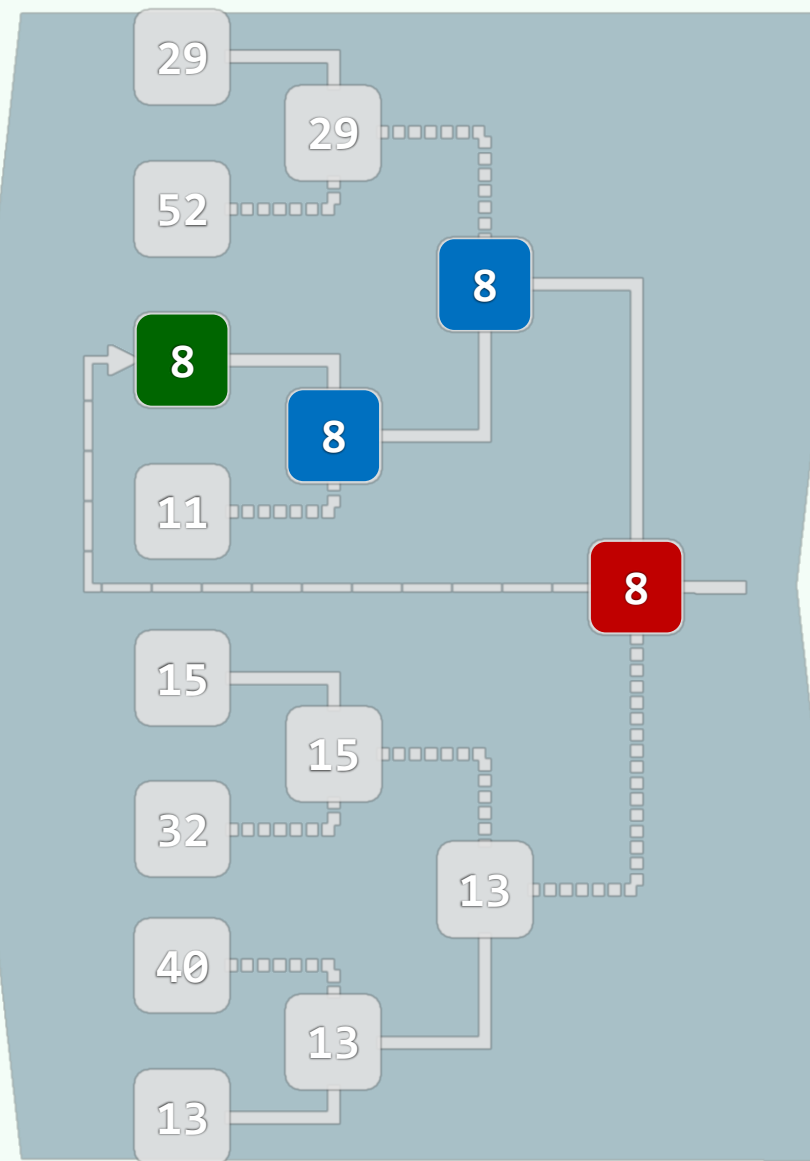
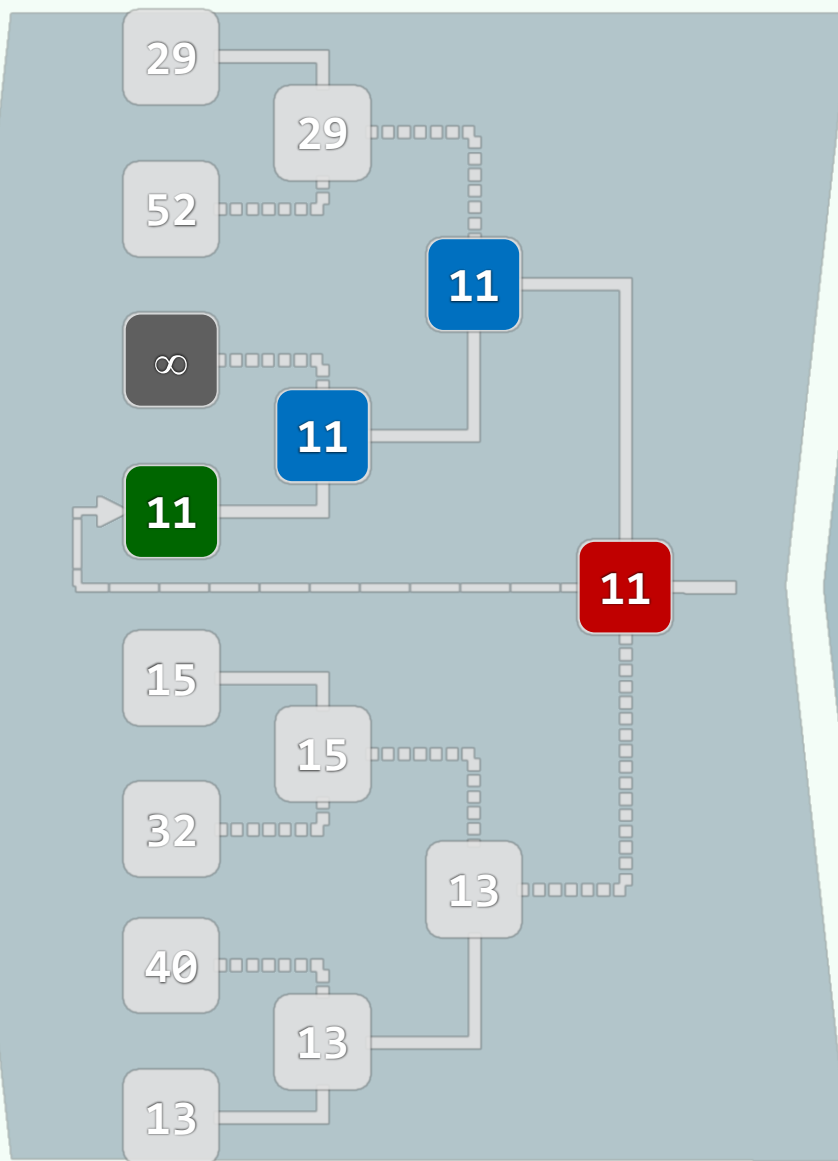
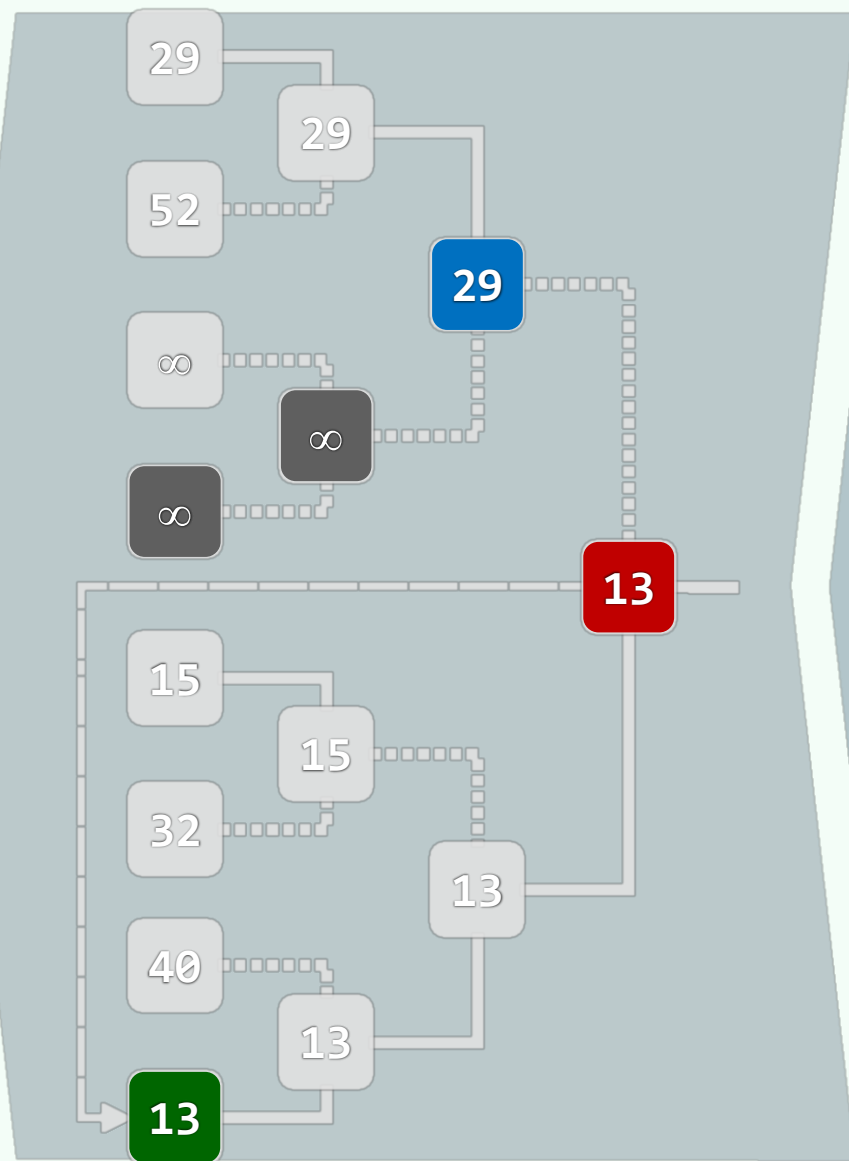
CREATE a tournament tree for the input list

while there are active leaves

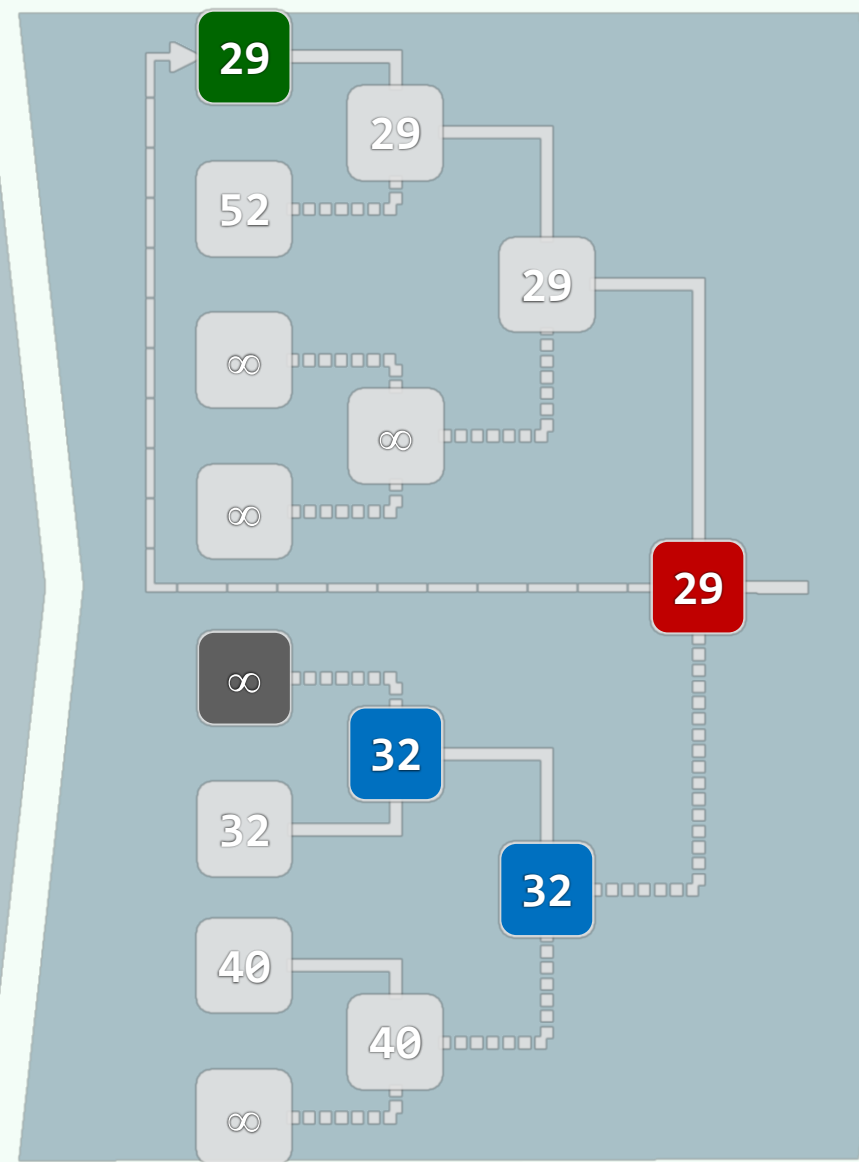
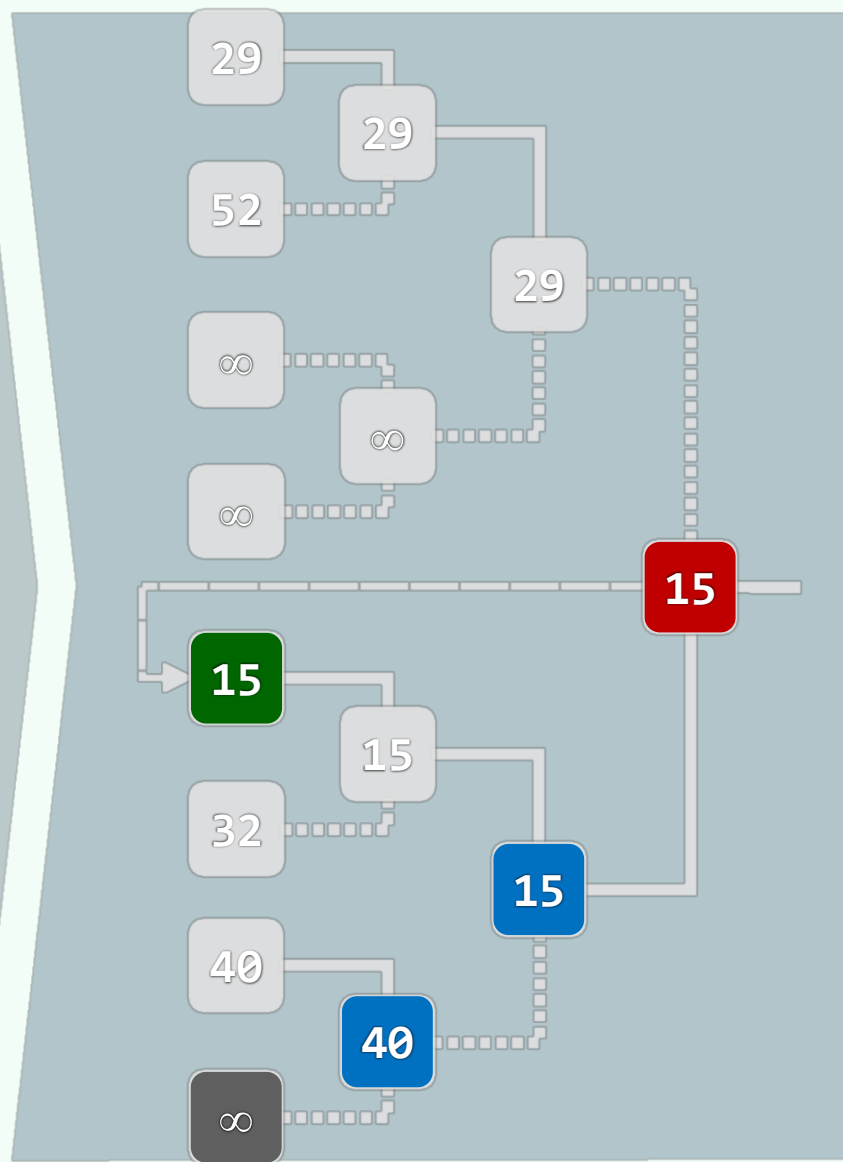
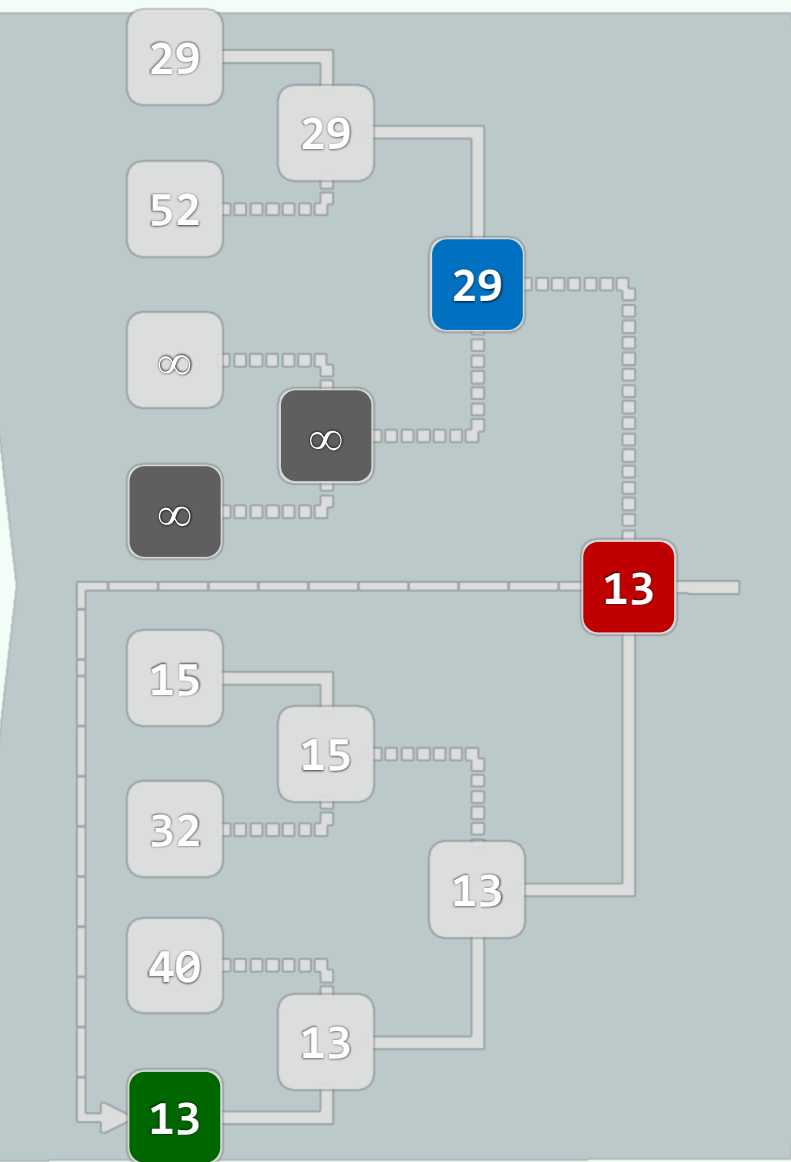
- **REMOVE** the root
- **RETRACE** the root down to its leaf
- **DEACTIVATE** the leaf
- **REPLAY** along the path back to the root



实例(1/2)



实例(2/2)



效率

❖ 空间: $O(\text{节点数}) = O(\text{叶节点数}) = O(n)$

❖ 构造: 仅需 $O(n)$ 时间

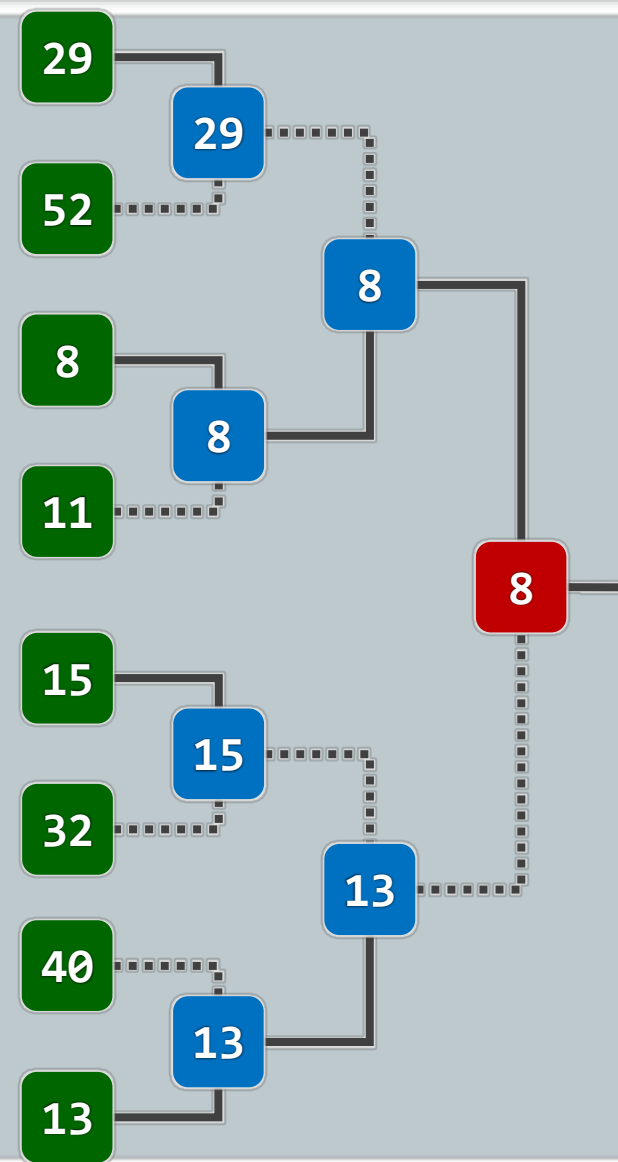
❖ 更新: 每次都须全体重赛 replay ?

唯上一优胜者的祖先, 才有必要!

❖ 为此 只需从其所在叶节点出发, 逐层上溯直到树根

如此 为确定各轮优胜者, 总共所需时间仅 $O(\log n)$

❖ 时间: $n \text{ 轮} \times O(\log n) = O(n \log n)$, 达到下界



选取

❖ 借助锦标赛树，从 n 个元素中找出最小的 k 个， $k \ll n$

- 初始化： $O(n)$ // $n-1$ 次比较
- 迭代 k 步： $O(k \cdot \log n)$ // 每步 $\log n$ 次比较

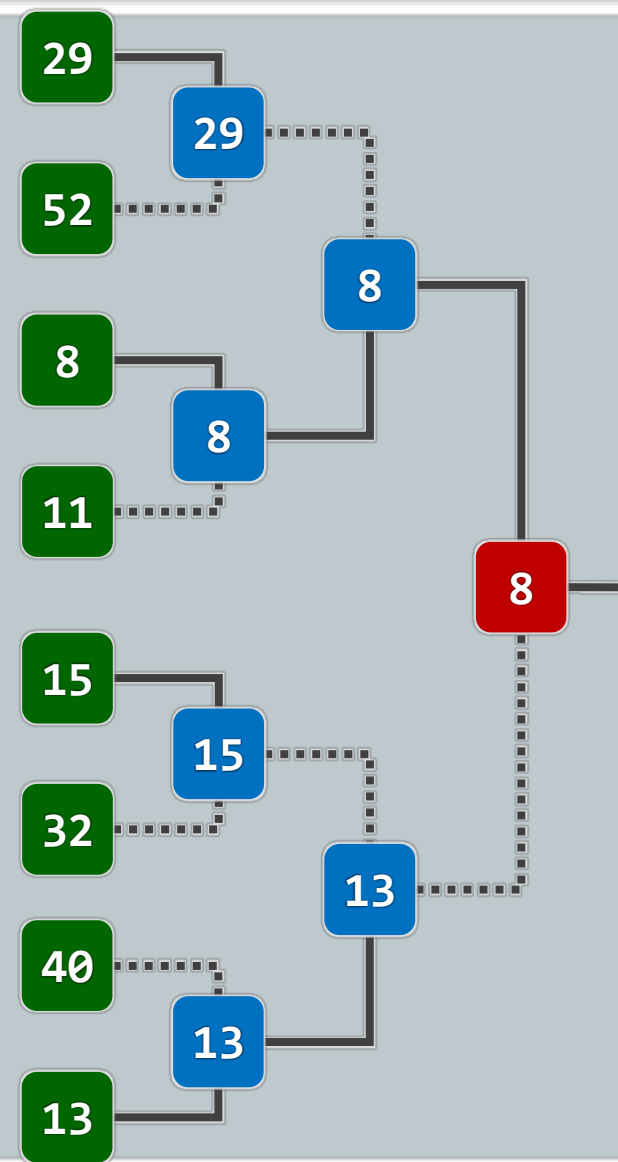
与小顶堆旗鼓相当？

❖ 渐近意义上，的确如此

但就常数而言，区别不小...

❖ Floyd算法、delMax()中的percolateDown()

在每一层需做2次比较，累计大致 $2 \cdot \log n$ 次



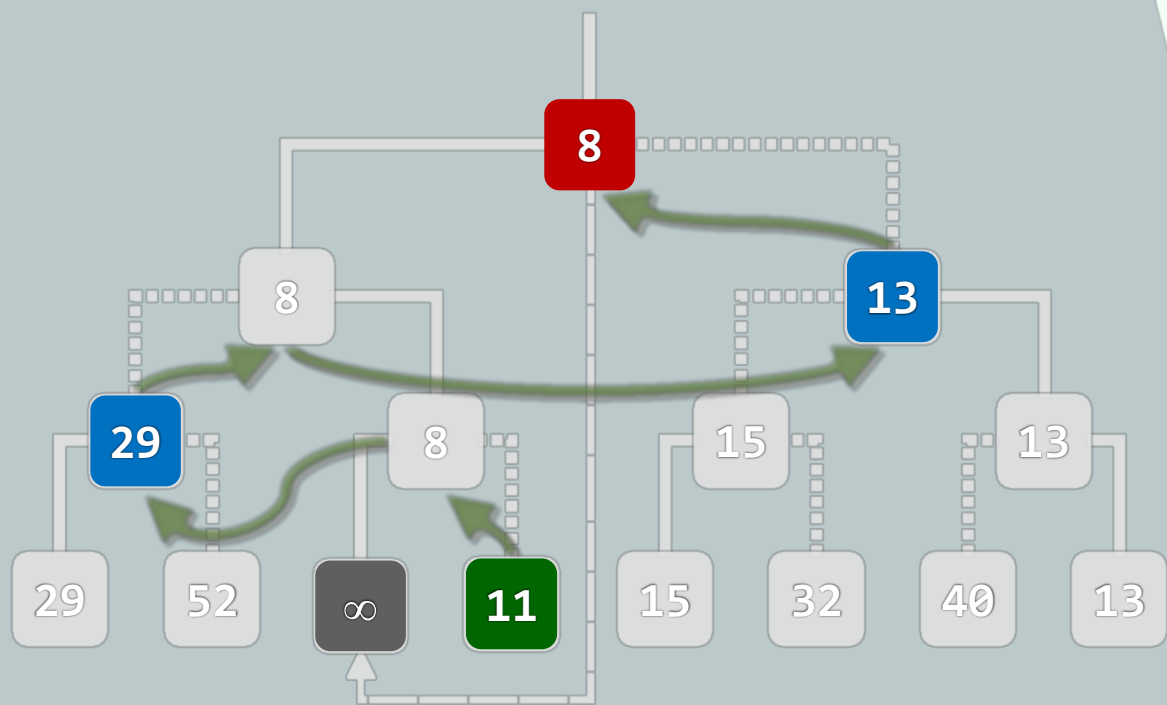
优先级队列

锦标赛树：败者树

结构 + 重赛算法

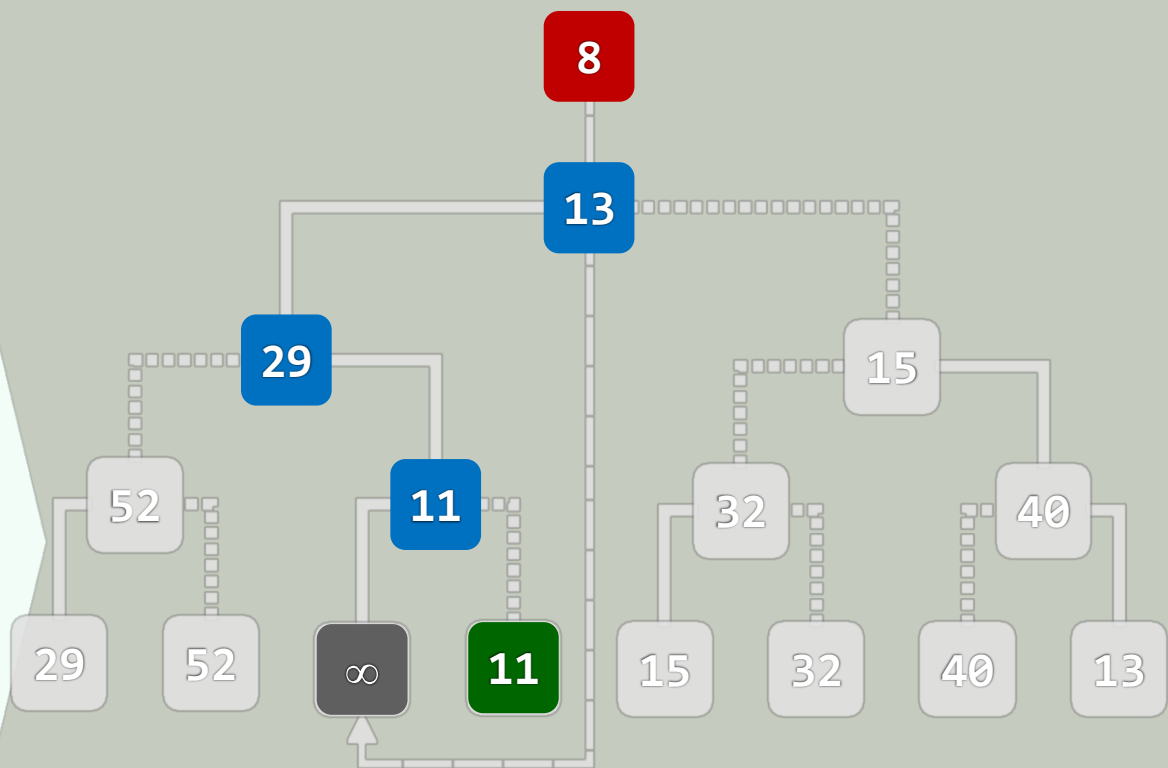
❖ 重赛过程中，须交替访问沿途节点及其兄弟

如何避免这类迂回？

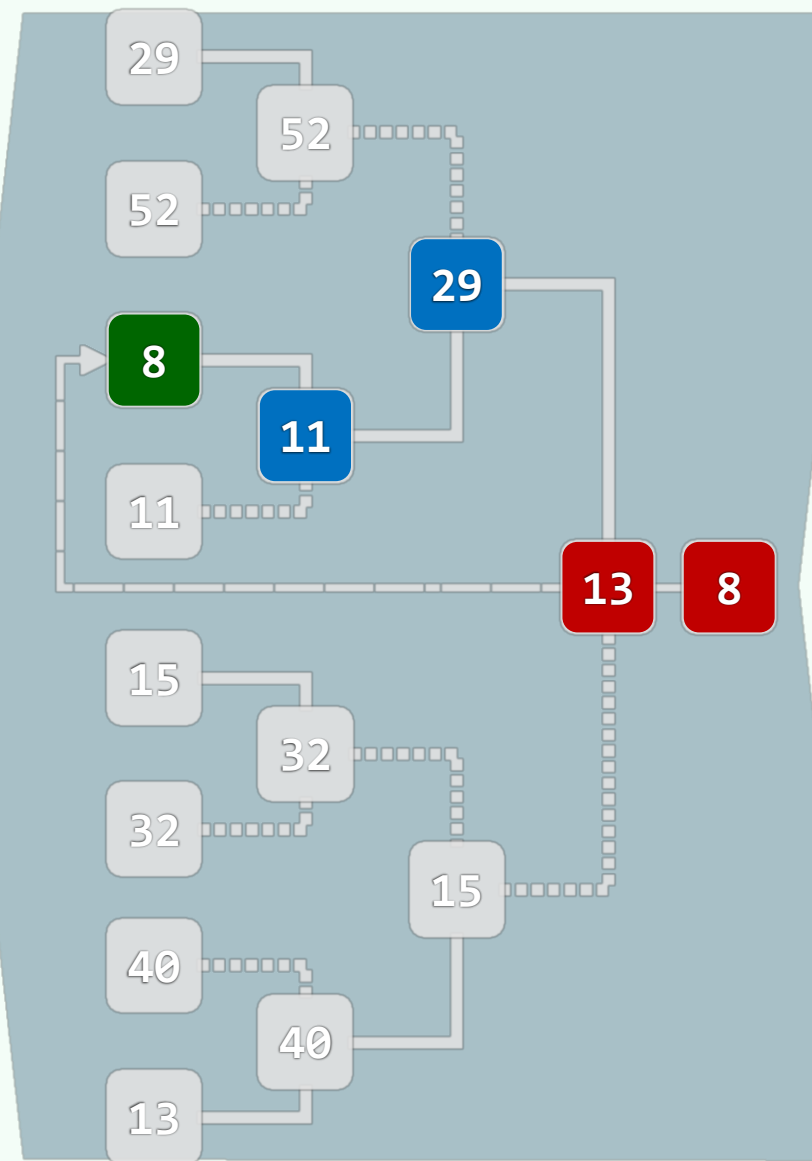
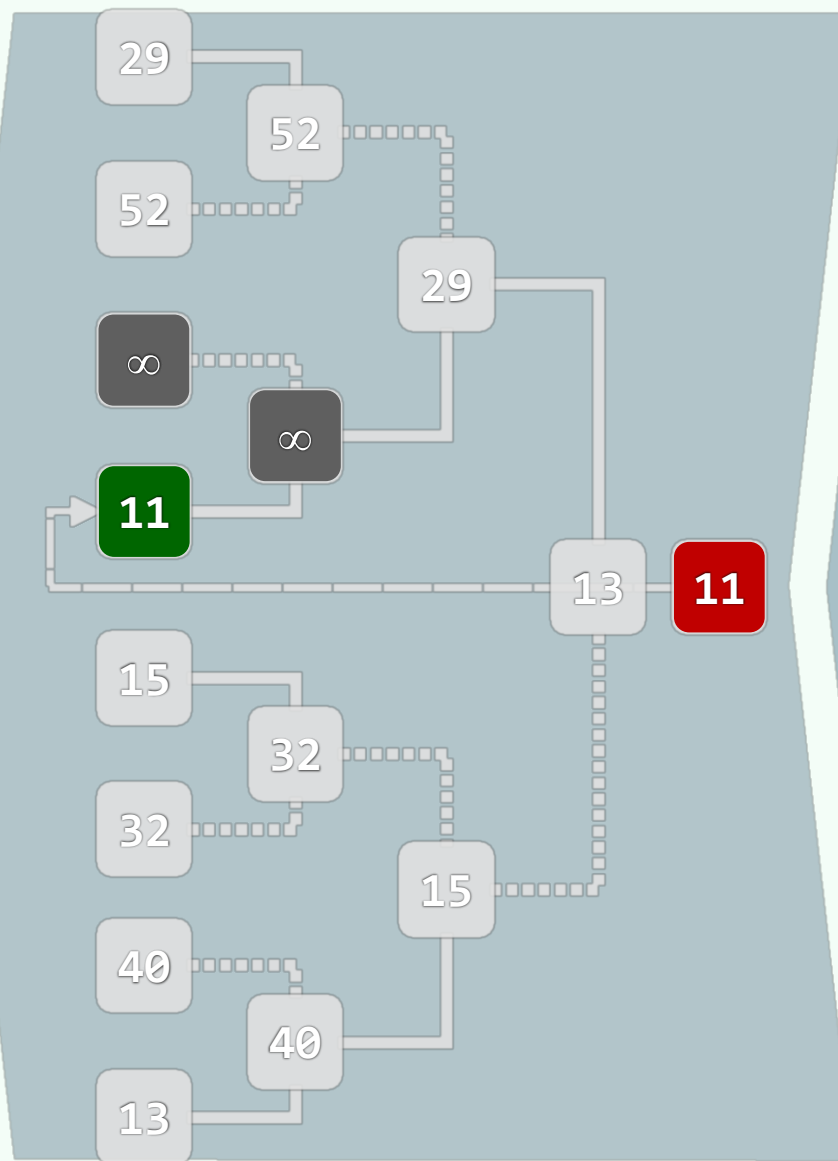
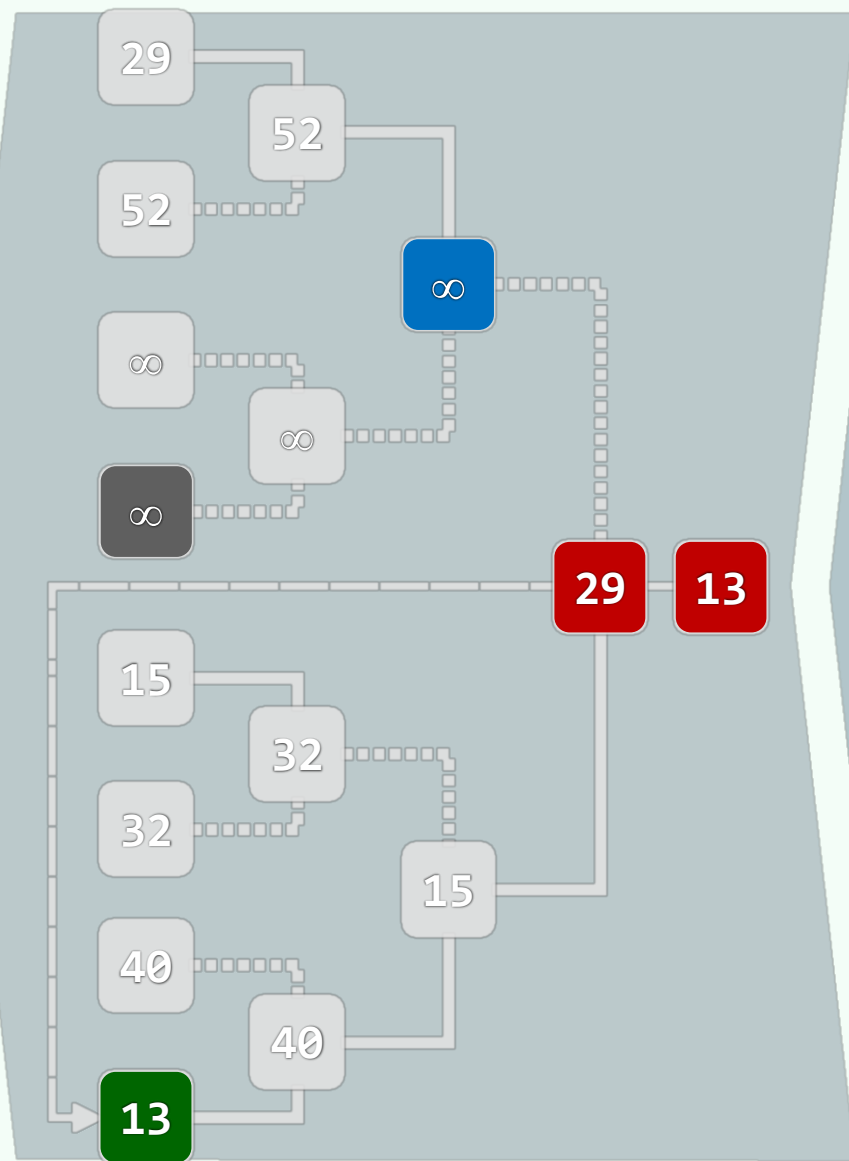


❖ 内部节点，记录对应比赛的败者

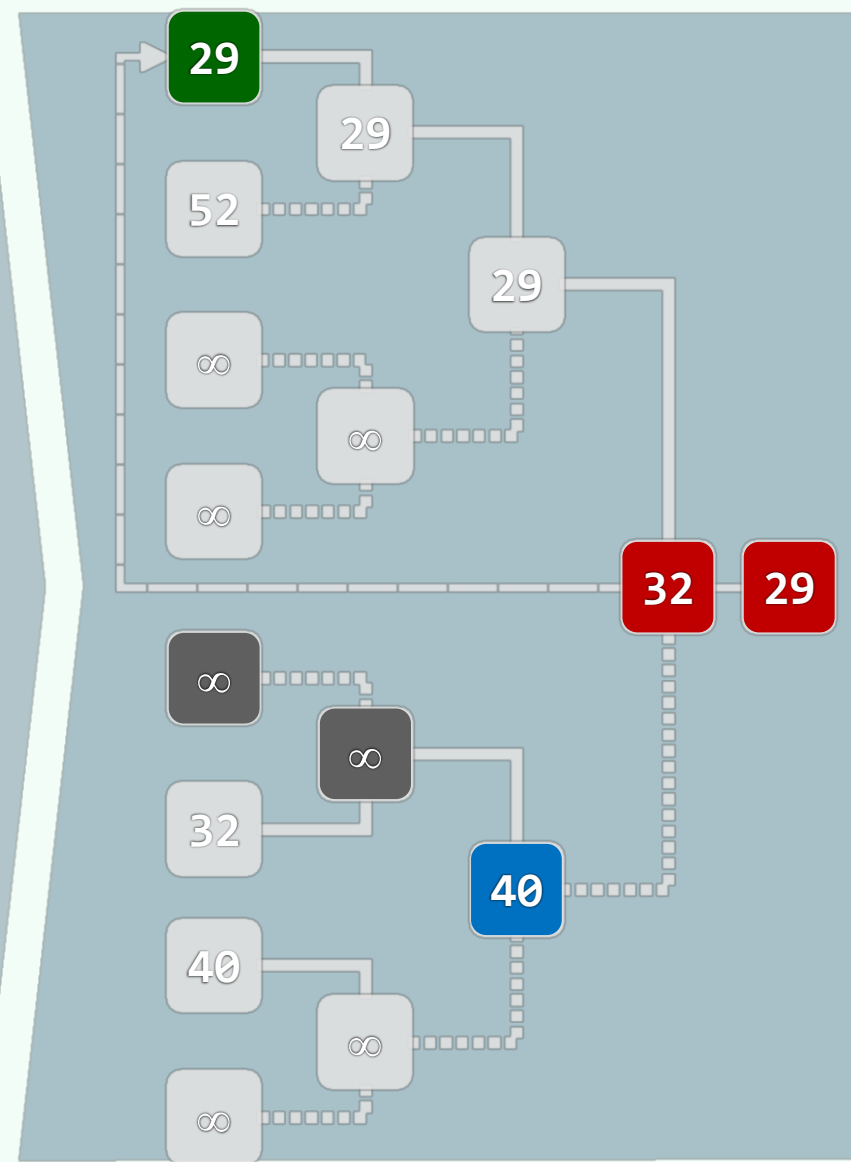
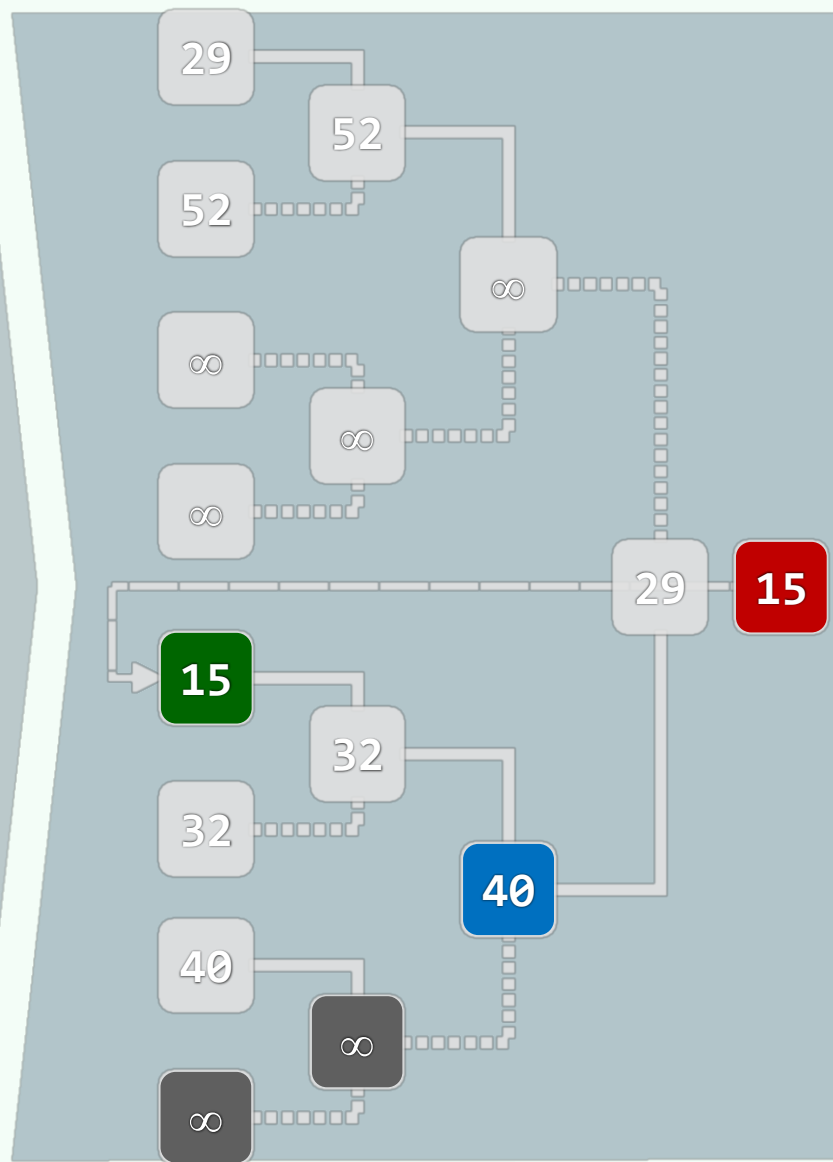
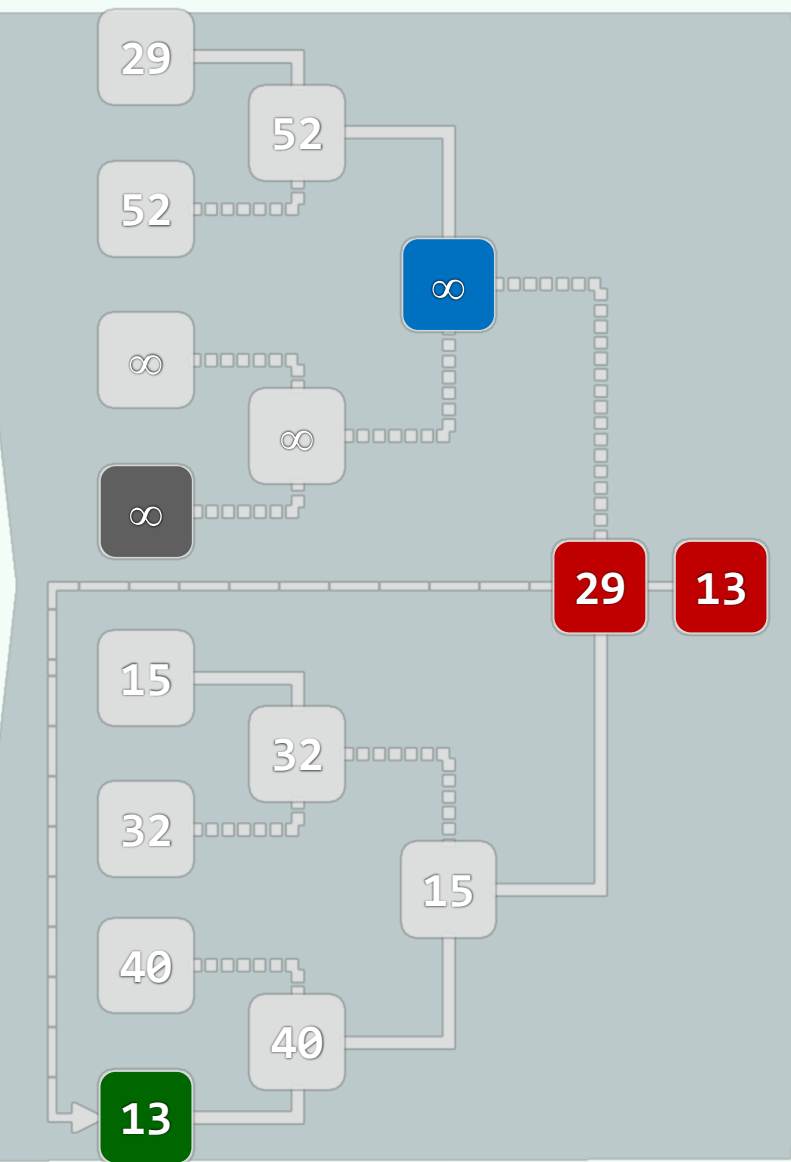
增设根的“父节点”，记录冠军



实例(1/2)



实例(2/2)



优先级队列

左式堆： 沿藤合并

堆合并

❖ $H = \text{merge}(A, B)$: 将堆A和B合二为一

//不妨设 $|A| = n \geq m = |B|$

❖ 方法一: $A.\text{insert}(B.\text{delMax}())$

$O(m * (\log m + \log(n + m)))$

$= O(m * \log(n + m))$

❖ 方法二: $\text{union}(A, B).\text{heapify}(n + m)$

$O(m + n)$

❖ 有没有更好的办法? 比如...

❖ 可否奢望在 $\dots O(\log n) \dots$ 时间内实现 $\text{merge}()$?



A

+



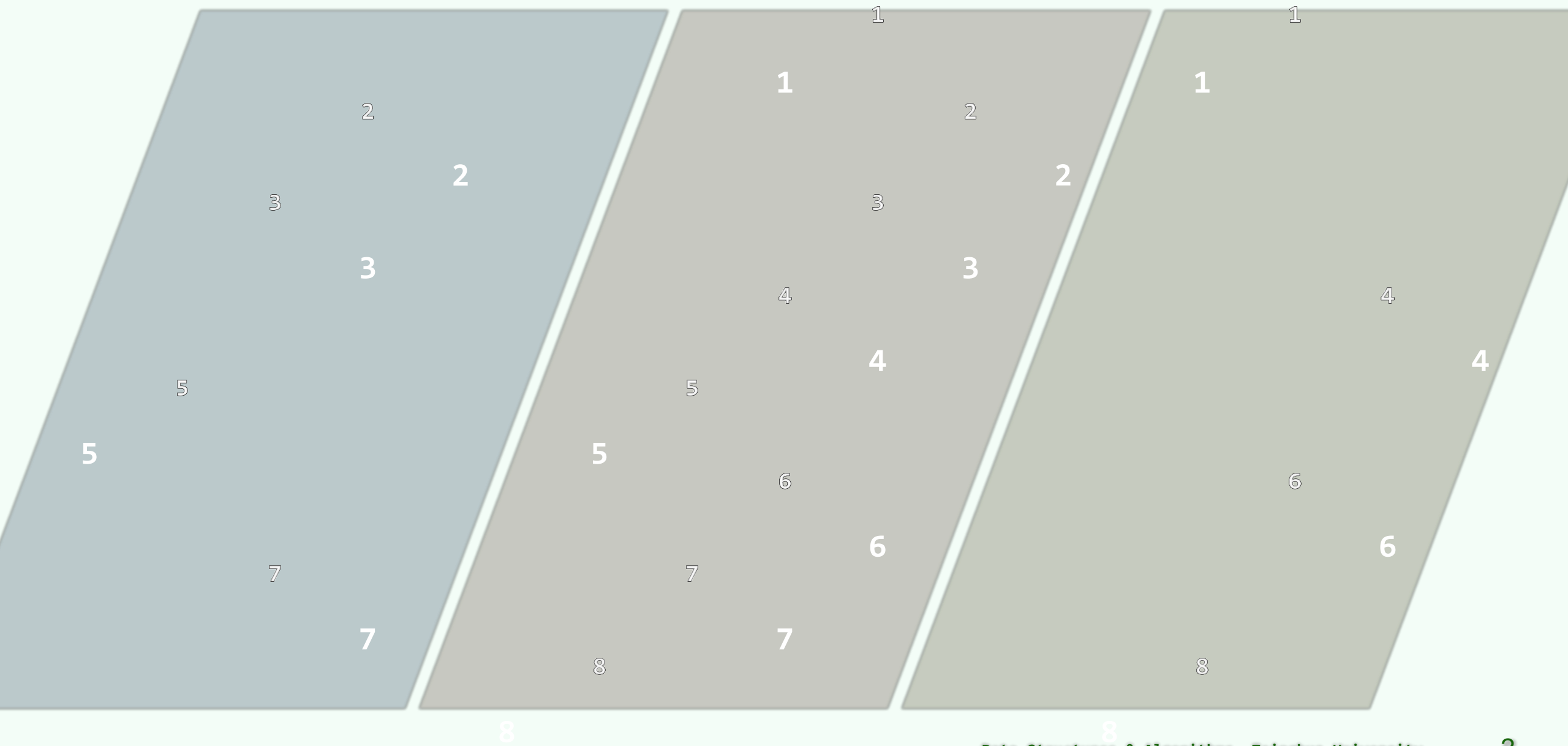
B

||

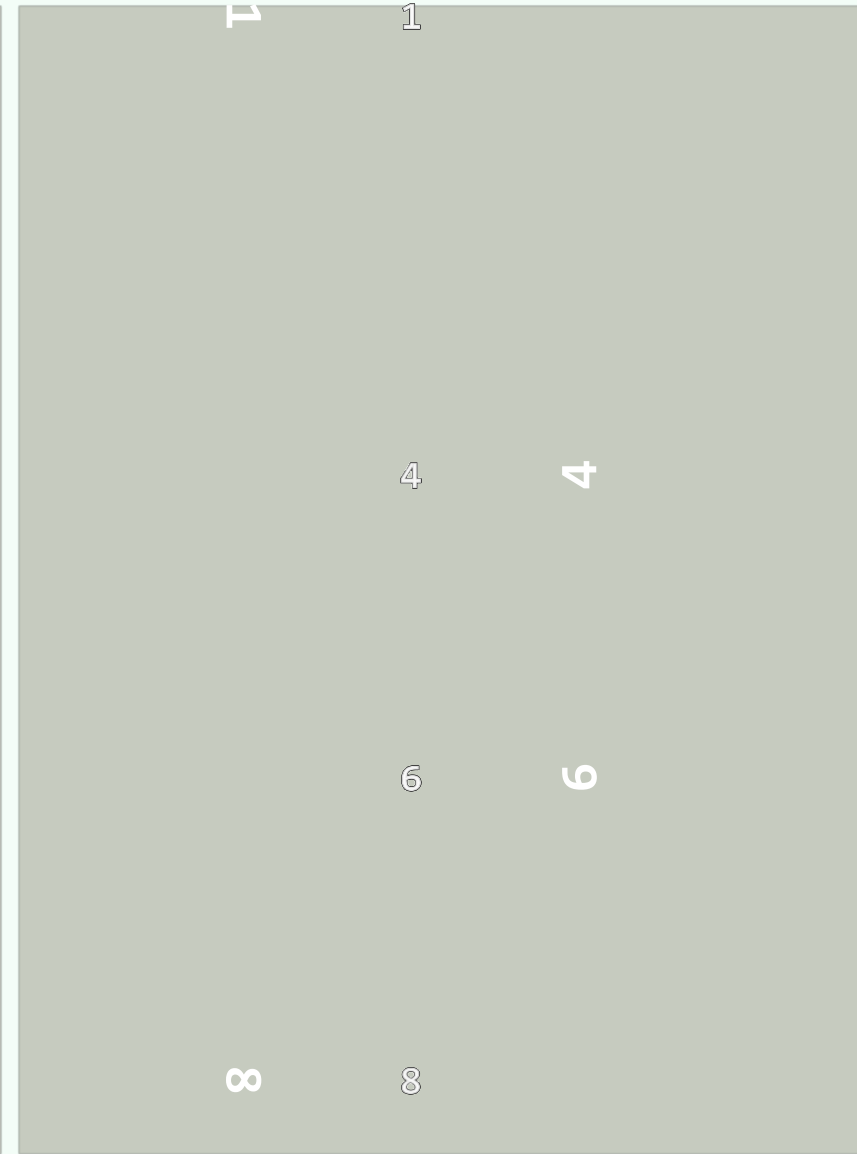
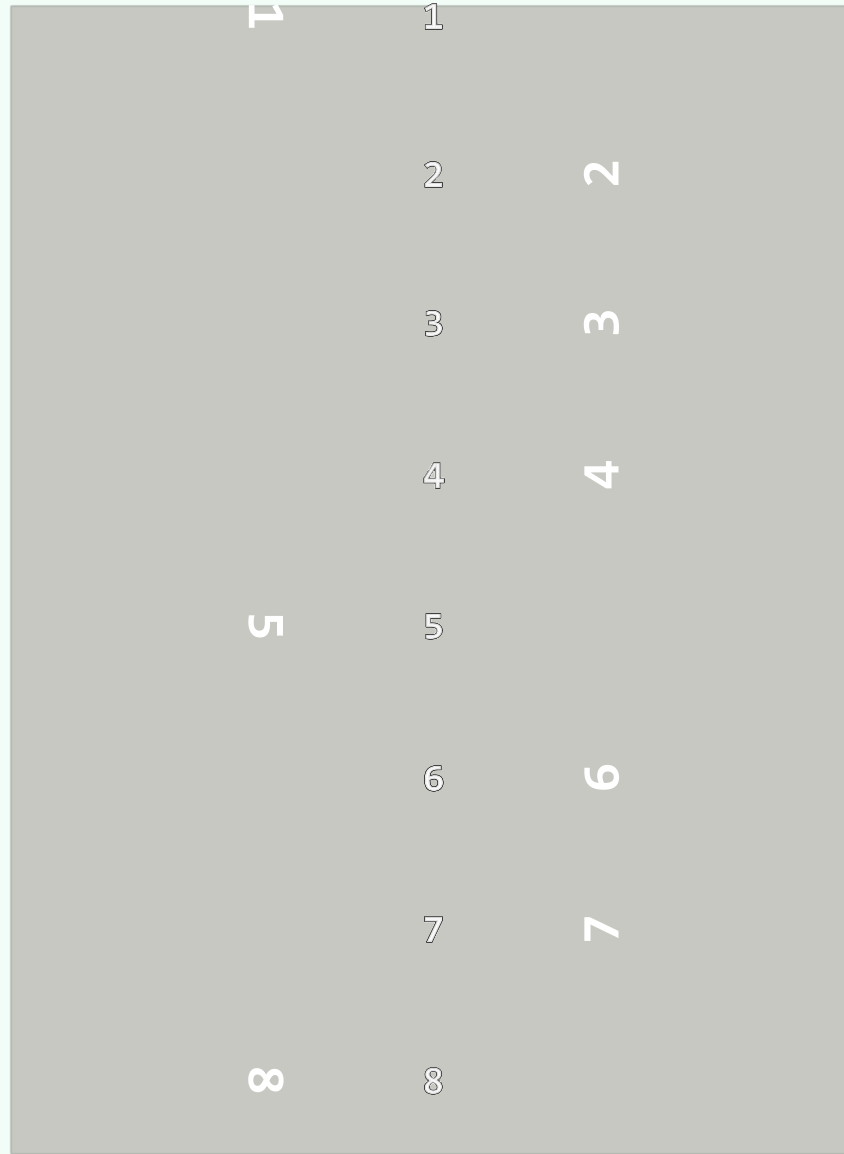
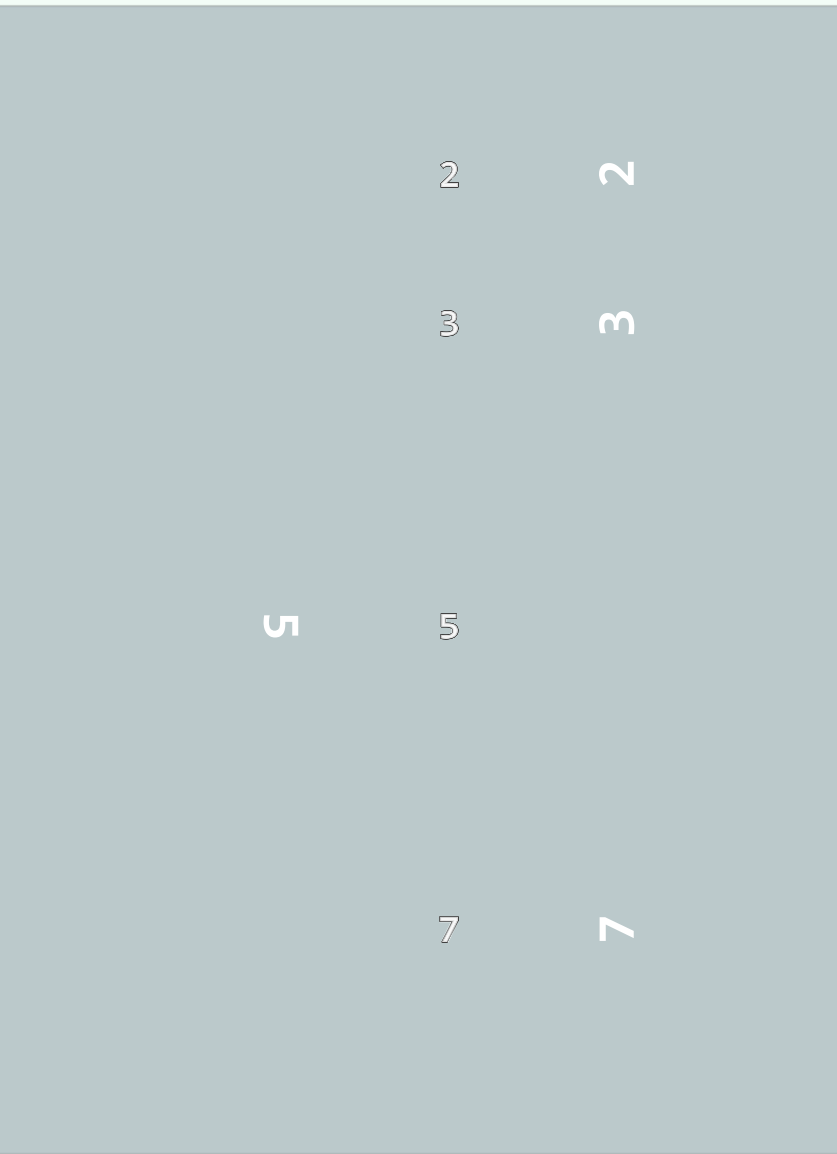


H

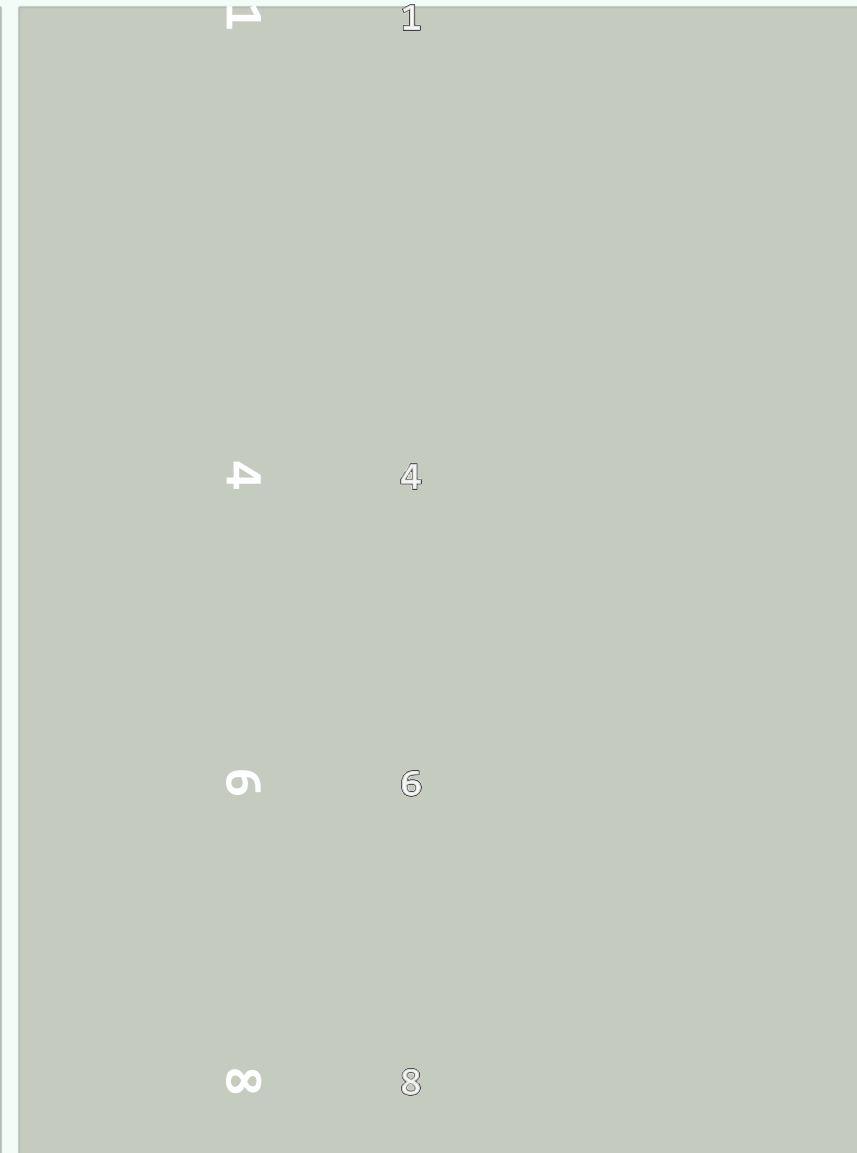
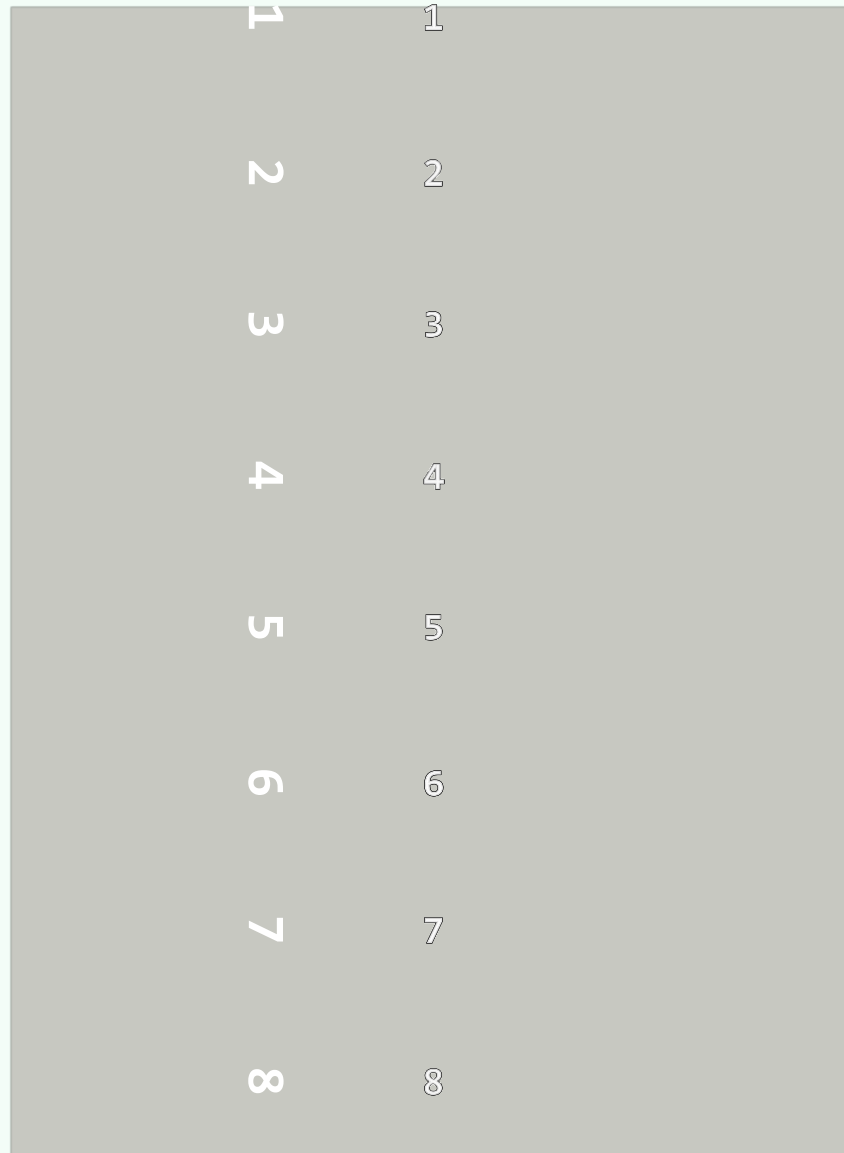
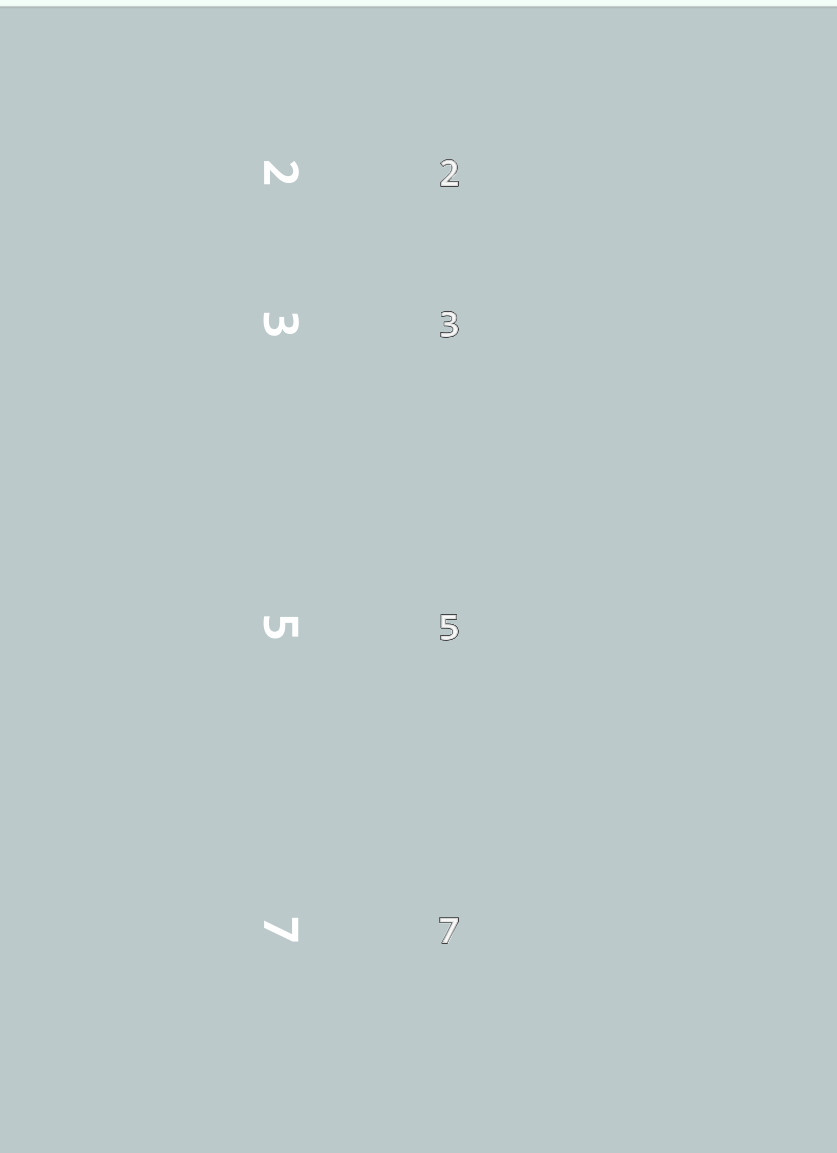
两堆合并 = 二路归并



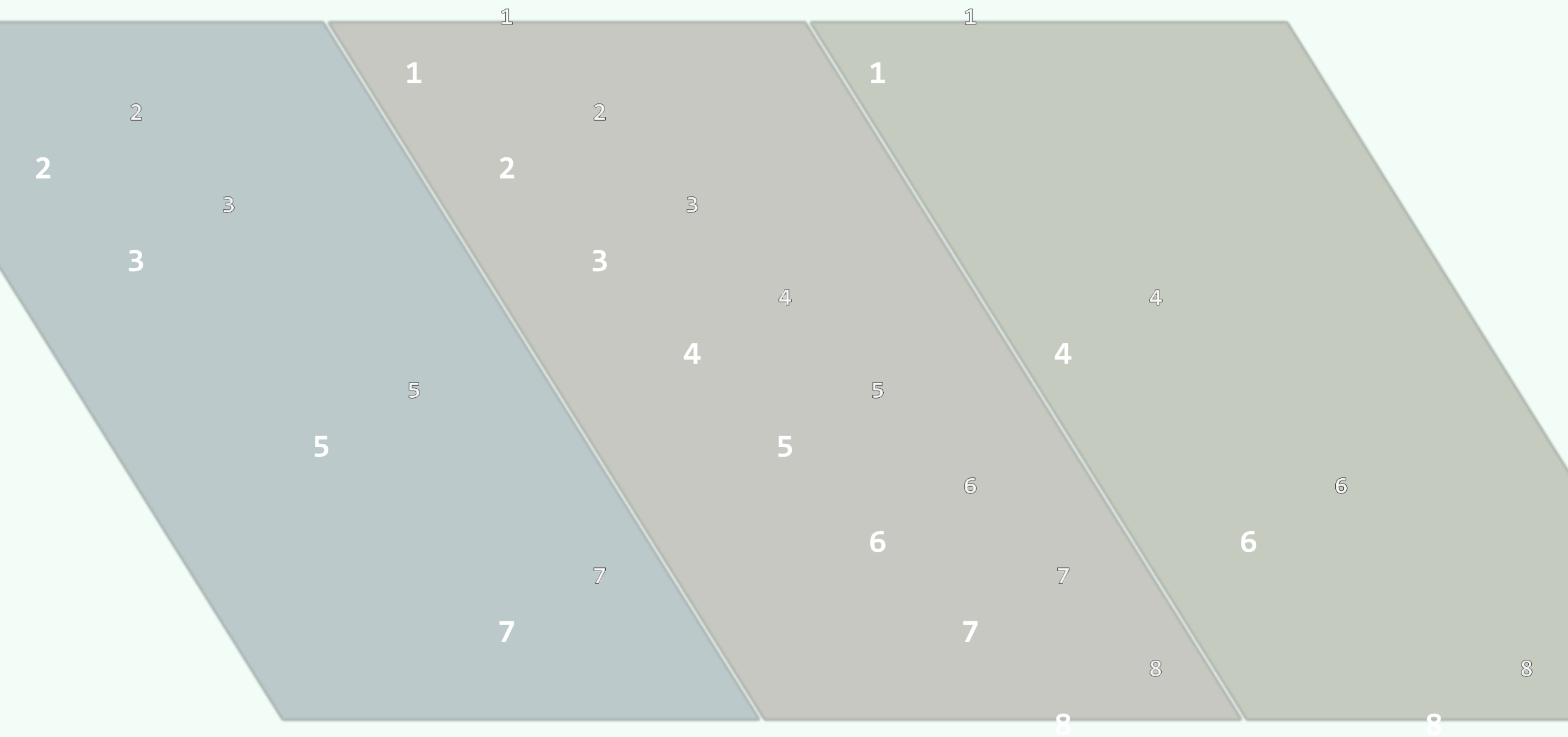
两堆合并 = 二路归并



简捷 = 统一沿右侧藤



简捷 = 统一沿右侧藤

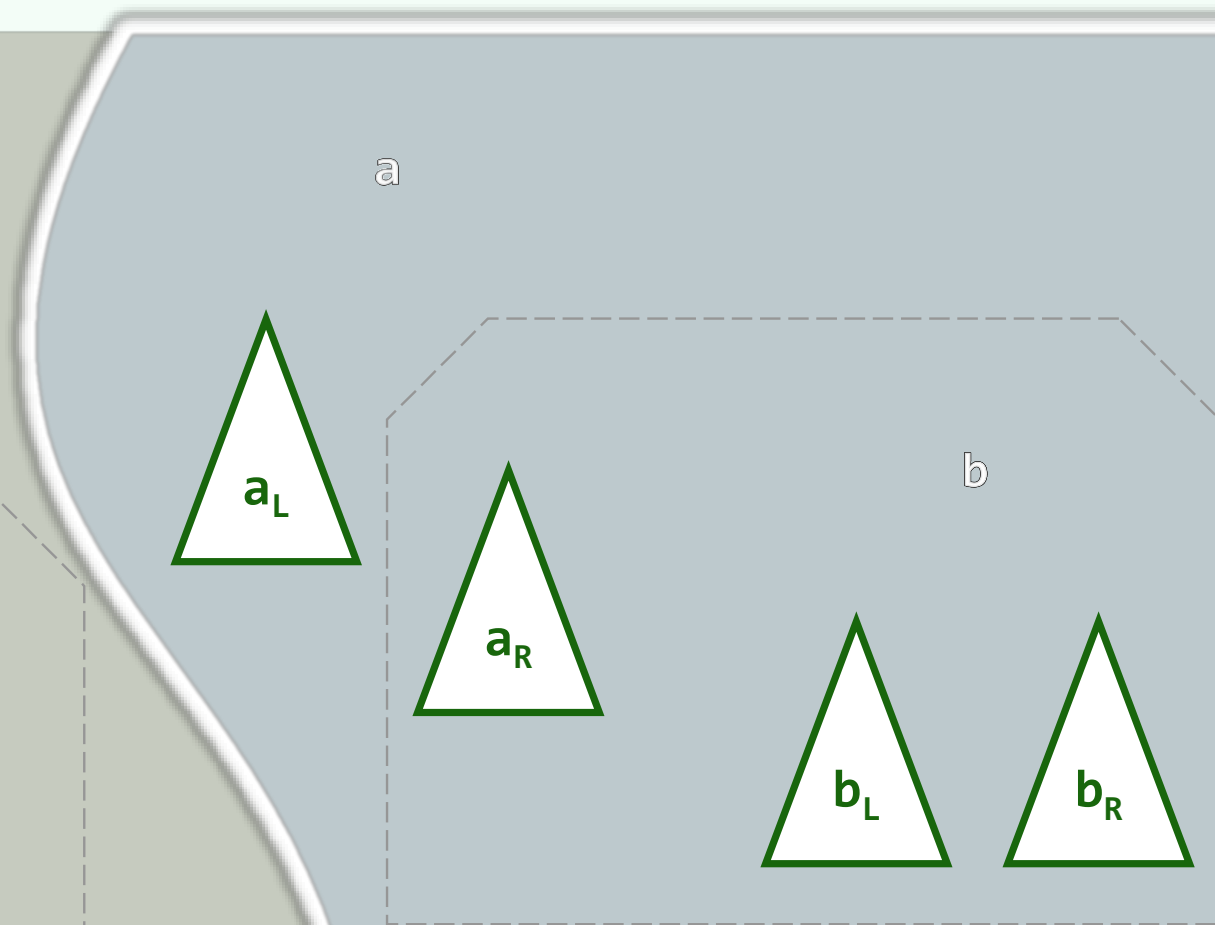
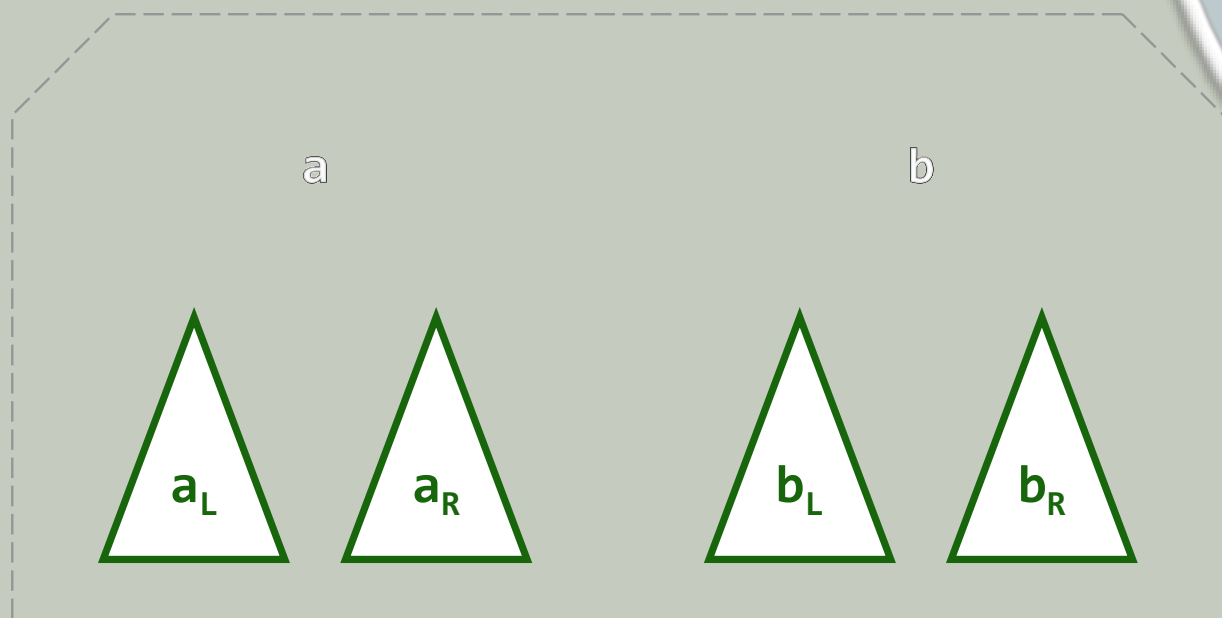


递归实现

所需时间 $\propto$ 右侧藤总长

~~$O(n)$~~

$O(\log n)$



如何...控制藤长以...持续合并?

优先级队列

左式堆： NPL与控制藤长

可持续 = 单侧倾斜

❖ C. A. Crane, 1972:

保持堆序性，附加新条件，使得

在堆合并过程中，只涉及少量节点： $O(\log n)$

❖ 新条件 = 单侧倾斜：

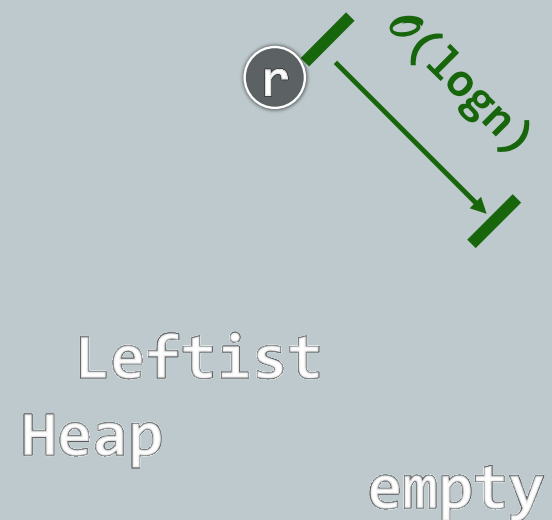
节点分布偏向于左侧

合并操作只涉及右侧

❖ 可是，果真如此，则拓扑上...

不见得是完全二叉树，结构性无法保证！？

❖ 是的，实际上，结构性并非堆结构的本质要求



空节点路径长度

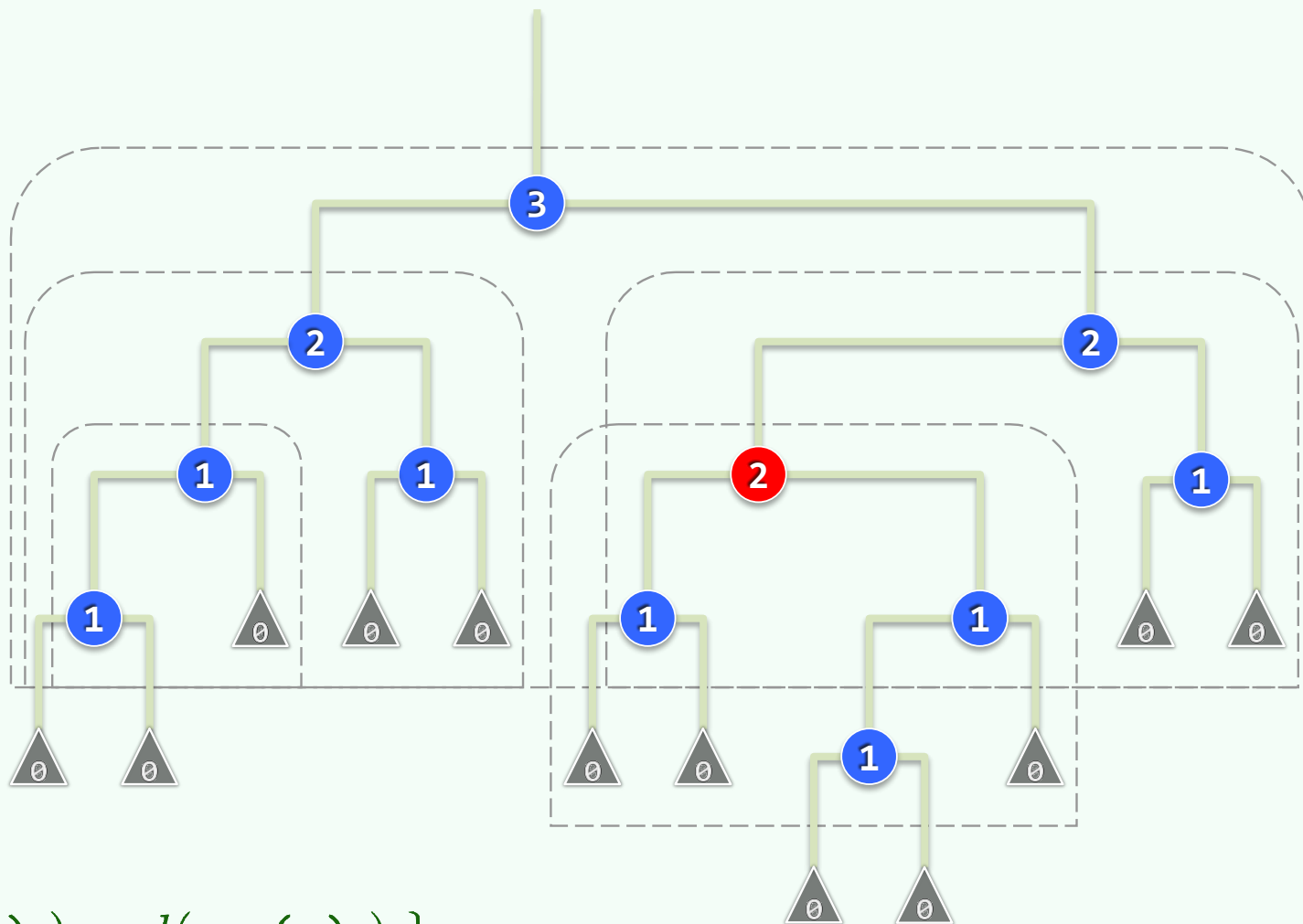
❖ 引入所有的外部节点

- 消除一度节点
- 转为真二叉树

❖ Null Path Length

- $npl(NULL) = 0$
- $npl(x) = 1 + \min\{ npl(lc(x)), npl(rc(x)) \}$

❖ 验证: $npl(x)$ = x到外部节点的最近距离 = 以x为根的最大满子树的高度



左式堆 = 处处左倾

❖ 对任何内节点x, 都有:

$$npl(lc(x)) \geq npl(rc(x))$$

❖ 推论:

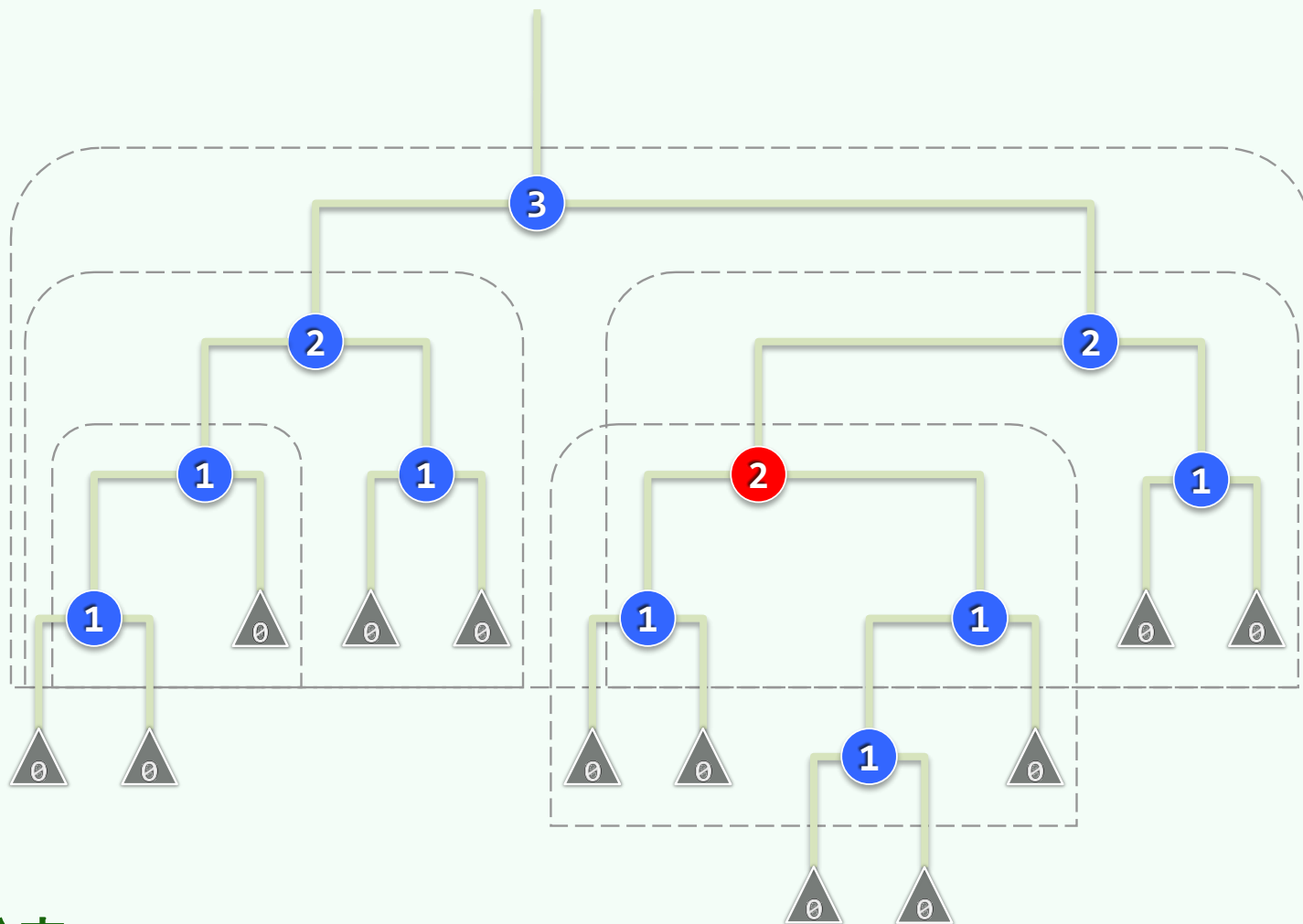
$$npl(x) = 1 + npl(rc(x))$$

❖ 左倾性与堆序性, 相容而不矛盾

❖ 左式堆的子堆, 必是左式堆

❖ 左式堆倾向于**更多**节点分布于左侧分支

❖ 这是否意味着, 左子堆的规模 and 高度必然大于右子堆?



右侧链

- ❖ $rChain(x)$: 从节点 x 出发, 一直沿**右分支**前进
- ❖ 特别地, $rChain(r)$ 的终点, 即全堆中**最浅**的外部节点

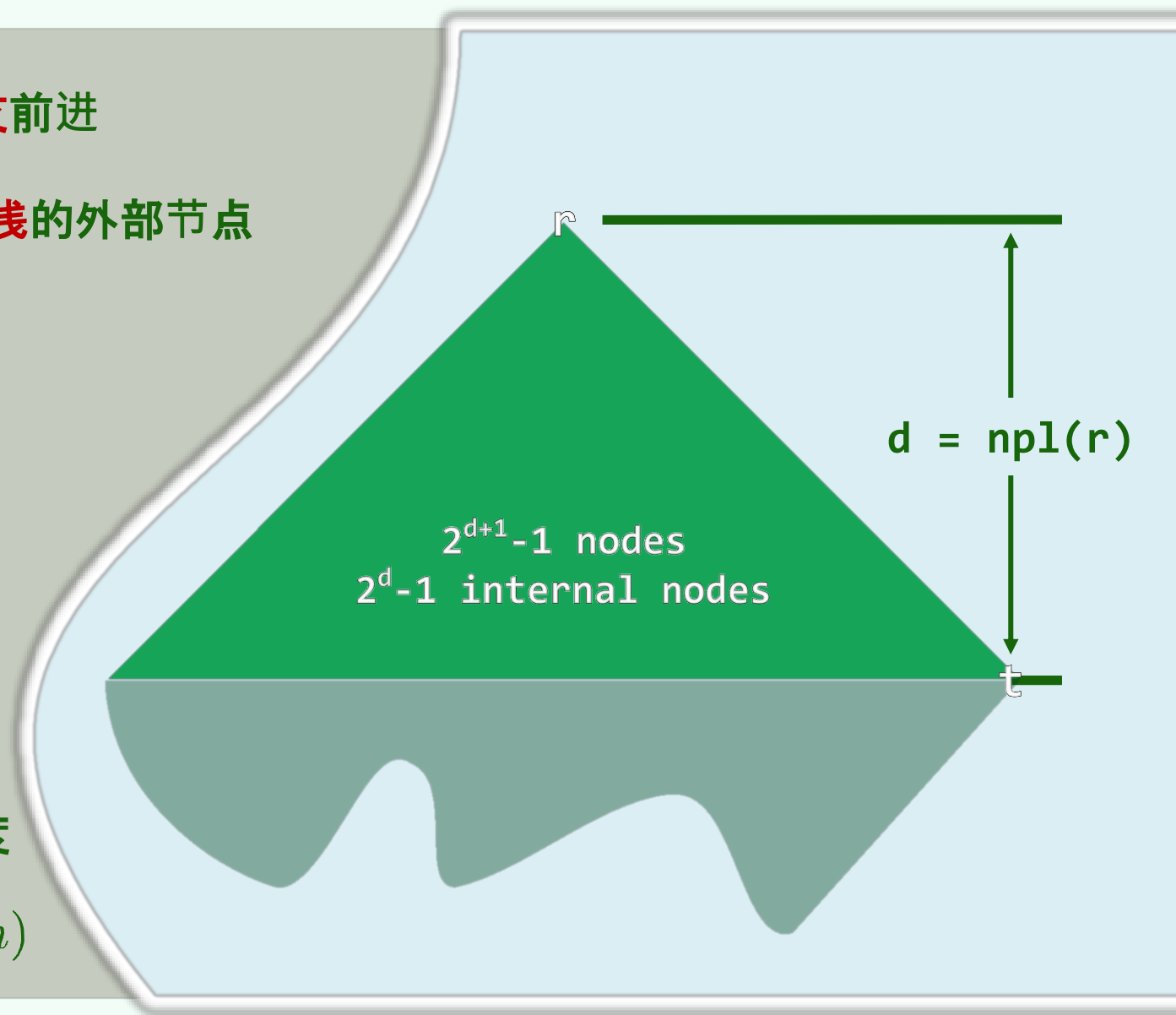
- $npl(r) \equiv |rChain(r)| = d$
- 存在一棵以 r 为根、高度为 d 的满子树

- ❖ 右侧链长为 d 的左式堆, **至少**包含

- $2^d - 1$ 个内部节点
- $2^{d+1} - 1$ 个节点

- ❖ 反之, 包含 n 个节点的左式堆, 右侧链长度

$$d \leq \lfloor \log_2(n + 1) \rfloor - 1 = \mathcal{O}(\log n)$$



优先级队列

左式堆： 合并算法

LeftHeap

```
template <typename T> //基于二叉树，以左式堆形式实现的优先级队列
```

```
class PQ_LeftHeap : public PQ<T>, public BinTree<T> {
```

```
public:  T getMax() { return _root->data; }
```

```
void insert(T); T delMax(); //均基于统一的合并操作实现...
```

```
PQ_LeftHeap( PQ_LeftHeap & A, PQ_LeftHeap & B ) {
```

```
    _root = merge(A._root, B._root); _size = A._size + B._size;
```

```
    A._root = B._root = NULL; A._size = B._size = 0;
```

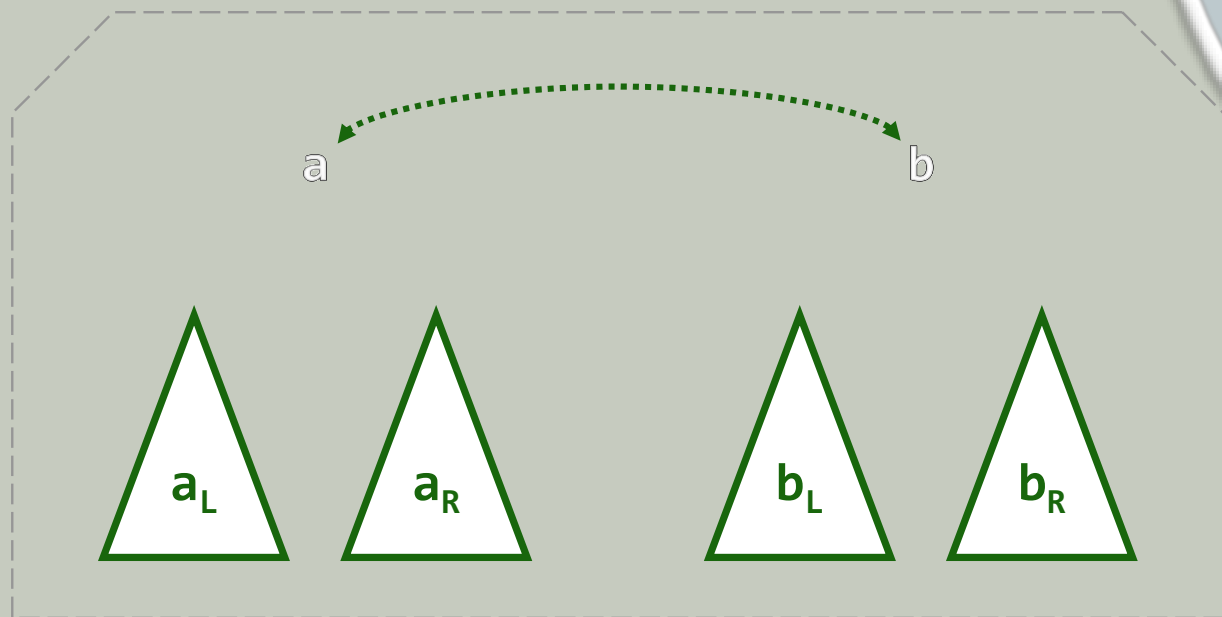
```
}
```

```
};
```

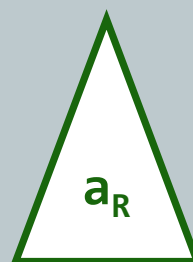
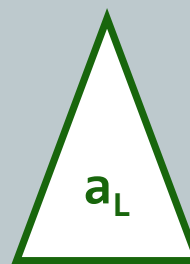
```
template <typename T> static BinNodePosi<T> merge(BinNodePosi<T>, BinNodePosi<T>);
```


递归：前处理 + 后处理

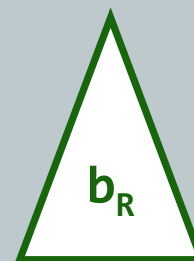
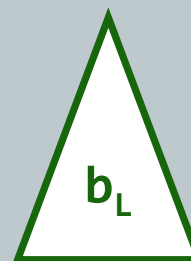
Before:
compare priority &
swap if necessary



a



b



After:
compare NPL &
flip if necessary

实现

```
template <typename T>

static BinNodePosi<T> merge( BinNodePosi<T> a, BinNodePosi<T> b ) {

    if ( !a ) return b; if ( !b ) return a; //递归基

    if ( lt( a->data, b->data ) ) swap( b, a ); //确保a不小

    ( a->rc = merge( a->rc, b ) )->parent = a; //将a的右子堆, 与b合并

    if ( ! a->lc || a->lc->npl < a->rc->npl ) //若有必要

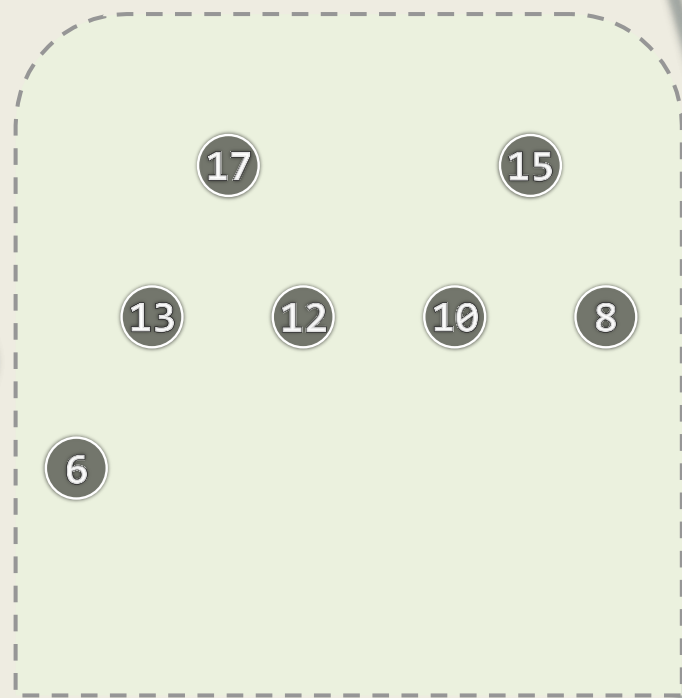
        swap( a->lc, a->rc ); //交换a的左、右子堆, 以确保左子堆的npl不小

    a->npl = a->rc ? 1 + a->rc->npl : 1; //更新a的npl

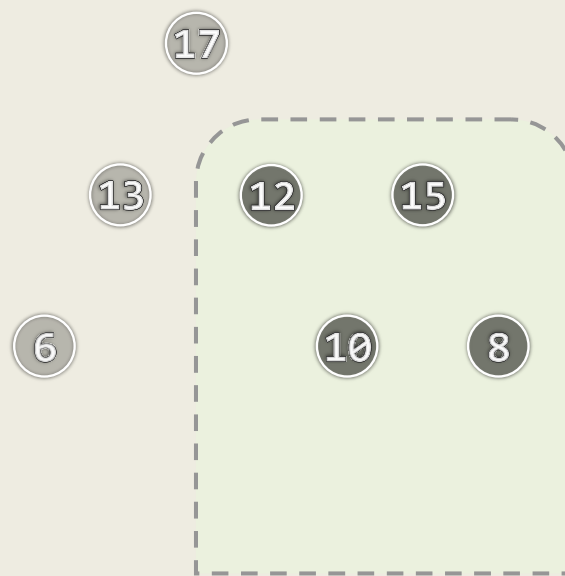
    return a; //返回合并后的堆顶

}
```

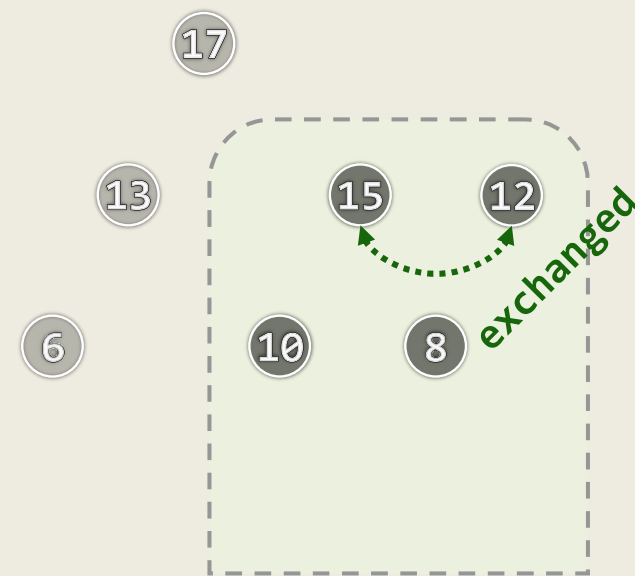
实例(1/5)



(a)

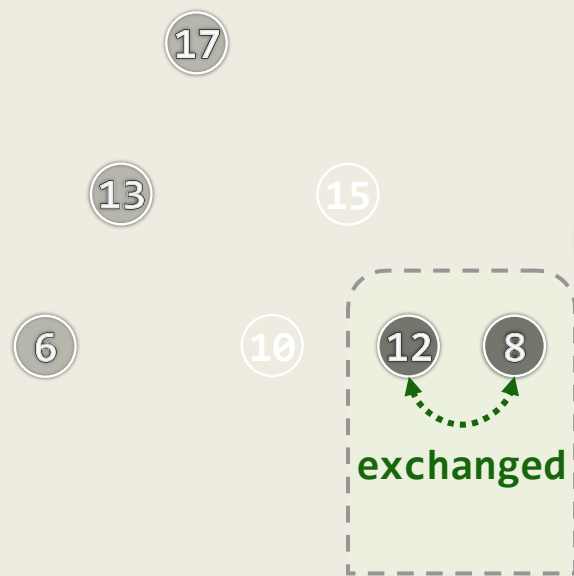


(b)

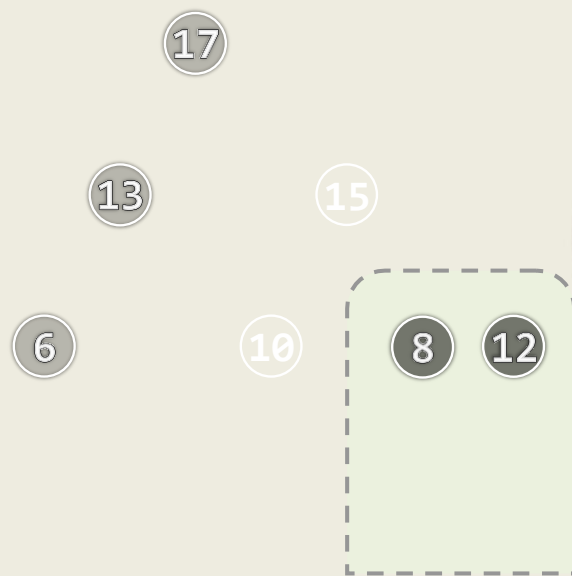


(c)

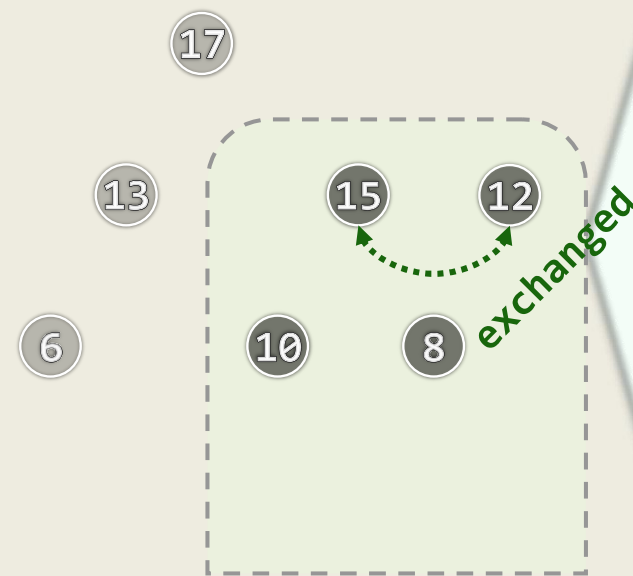
实例(2/5)



(e)

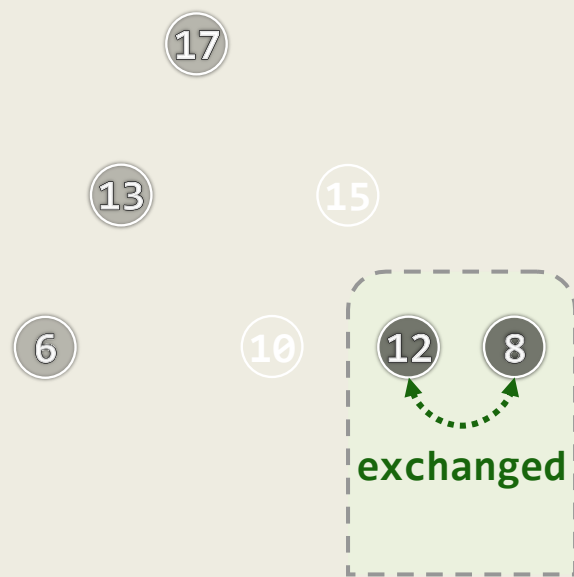


(d)

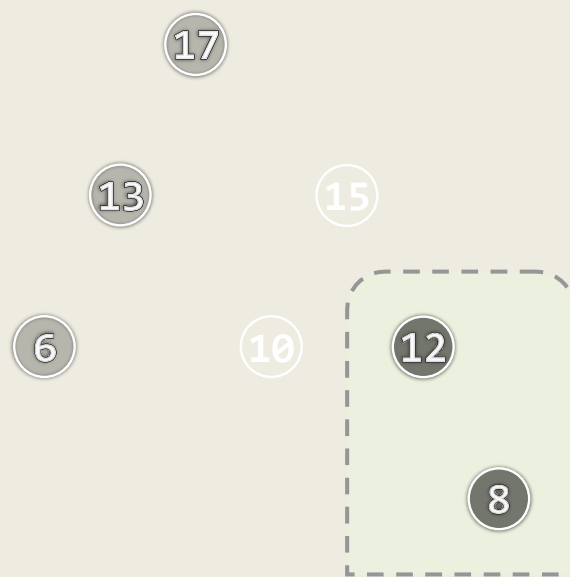


(c)

实例(3/5)



(e)

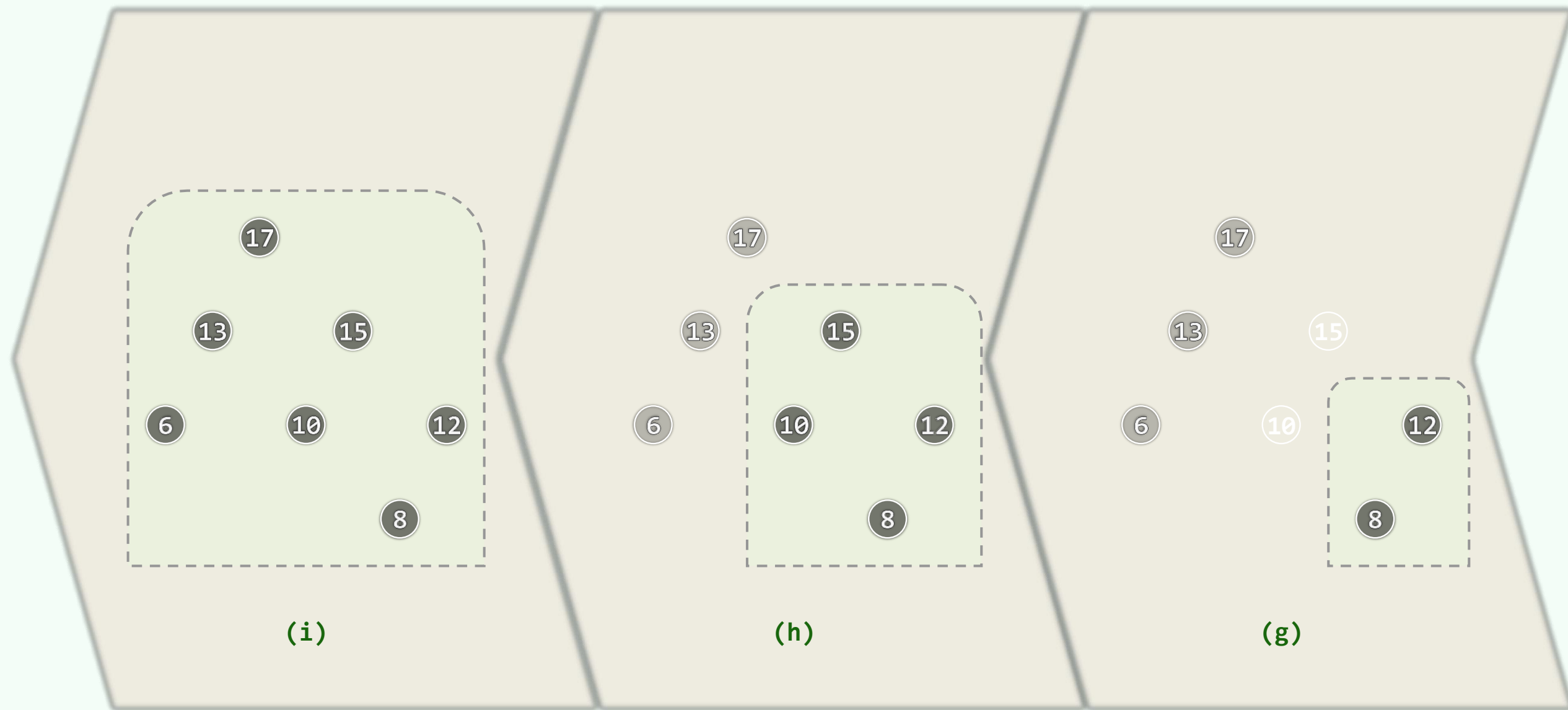


(f)

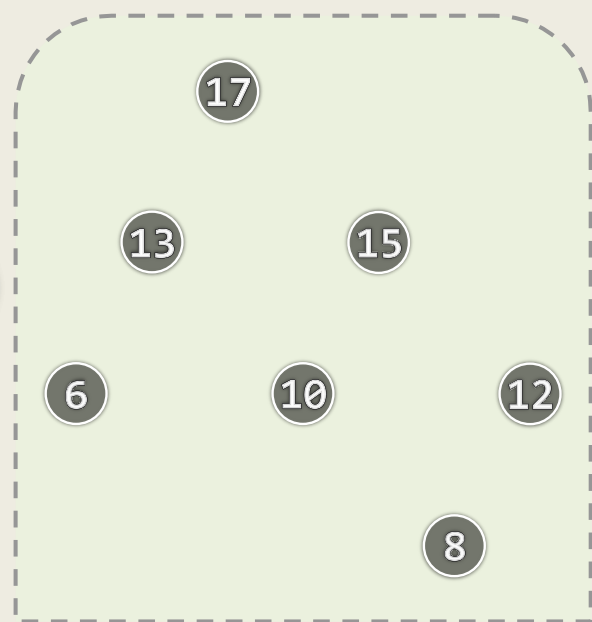


(g)

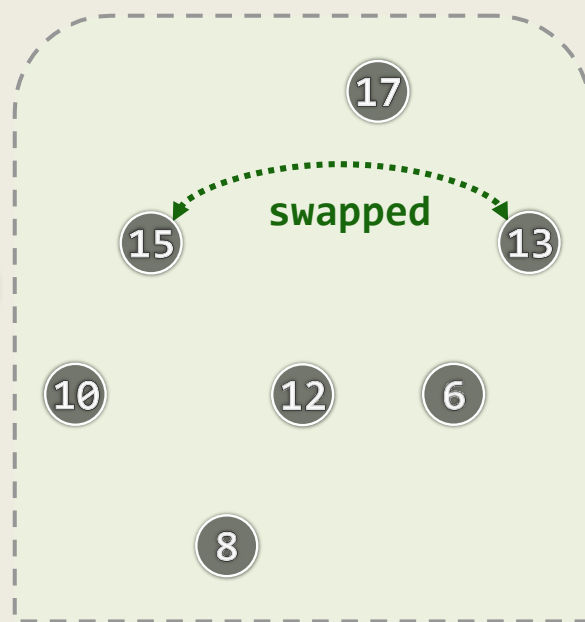
实例(4/5)



实例(5/5)



(i)



(j)



(k)

优先级队列

左式堆：插入与删除

插入

```
template <typename T> void PQ_LeftHeap<T>::insert( T e ) { //O(logn)
```

```
    _root = merge( _root, new BinNode<T>( e, NULL ) );
```

```
    _size++;
```

```
}
```

r  e

删除

```
template <typename T> T PQ_LeftHeap<T>::delMax() { //O(logn)

    BinNodePosi<T> lHeap = _root->lc; if (lHeap) lHeap->parent = NULL;

    BinNodePosi<T> rHeap = _root->rc; if (rHeap) rHeap->parent = NULL;

    T e = _root->data;

    delete _root; _size--;

    _root = merge( lHeap, rHeap );

    return e;

}
```

